

Apunte de Arquitectura de Computadoras



UTN FRRE

2015

Compilación, Formato y Algunos Textos

Originales: Albiach Matias, Alejandro.

Agradecimiento Especial: Juan Carcaño, Dante

Sack, Rodrigo Cerdan.

Apunte realizado siguiendo el programa analítico con las filminas, la bibliografía y la guía de autoestudio. Se toma el libro de Meinadier como base.

Pensado más como guía rápida o material de repaso que material único de estudio.

(Primer Release, Hasta ALU)

Unidad 1: Introducción a la Arquitectura de Computadoras

Sistema de Información: Es un subsistema de la organización. Elabora datos útiles (información) para la toma de decisiones.

Procesamiento de Datos: Pasos necesarios para obtener información (disminuye la incertidumbre) a partir de datos (representación en un conjunto de símbolos de hechos, atributos, valores).

Sistema Digital: Es un dispositivo o una combinación de dispositivos diseñado para generar, transmitir, procesar o almacenar información representada por señales digitales; es decir; que solo puedan tomar valores discretos. Un computador es un sistema digital.

Computador (Según Stallings):

Máquina digital electrónica programable para el tratamiento automático de la información, capaz de recibirla, operar sobre ella mediante procesos determinados y suministrar los resultados de tales operaciones.

Generaciones de Computadoras:

Generación	Años	Características	Resuelven
0	hasta 1945	Sistemas mecánicos y electro-mecánicos	No siguen instrucciones (ábacos, calculadoras mecánicas, etc.)
1	1945 – 1955	Tubos al vacío, tableros	Una sola tarea
2	1955 – 1965	Transistores y sistemas por lotes	Varias tareas
3	1965 – 1980	Circuitos integrados y multiprogramación	Varias tareas
4	desde 1980	Microprocesadores y Computadores personales	Varias tareas

Sistemas de Numeración: es un conjunto de símbolos y reglas de generación que permiten construir todos los números válidos.

- Binario
- Decimal
- Base M

Representación de los números en la máquina:

Todas las computadoras digitales actuales usan un lenguaje binario para representar la información internamente.

Operaciones con Números Binarios:

Suma:

0+0=0

0+1=1

1+0=1

1+1=10

Multiplicación:

0x0=0

0x1=0

1x0=0

1x1=1

Conversión de una base a decimal:

Numero en base b= $A_3A_2A_1A_0$

$$\text{Numero en Decimal} = \sum A_i * b^i$$

Ejemplo:

Numero=2301(4

Numero en decimal= $2*4^3+3*4^2+0*4^1+1$

Conversión de Decimal a una base:

Parte entera: Se divide el número por la base tantas veces como sea posible, luego los restos de las divisiones en orden inverso son los valores del número expresado en la base.

Ejemplo:

$177/4=44$ Resto=1

$44/4=11$ resto=0

$11/4=2$ resto=3

$2/4=0$ resto 2

Numero en base 4= 2301(4

Parte decimal: se multiplica la parte fraccionaria por la base tantas veces como sea necesario para obtener una parte fraccionaria igual a cero; la parte entera de los resultado se toma como dígito de la parte fraccionaria del número expresado en la nueva base, en el mismo orden de la multiplicación.

Ejemplo:

$0,625*2=1,250$

$0,250*2=0,500$

$0,500*2=1,000$

Parte fraccionaria en base 2=0,101

Complementos

Complemento Restringido (Complemento a 1, Comp. a la Base -1, Comp. n-1): En binario se obtiene el complemento restringido cambiando unos por ceros y viceversa.

$\text{CompRest}(a)=10^n-1-a$

Complemento Auténtico (Complemento a 2, Comp. a la Base, Comp. a n): El complemento a la base se obtiene sumando 1 al complemento restringido

$\text{Comp}(a)=10^n-a$

Aritmética Digital

Se utiliza el primer dígito binario para el signo, siendo 0 para números positivos y 1 para números negativos. Así pueden usarse 3 sistemas: El signo seguido del valor absoluto, el complemento a la base y el complemento a la base -1.

En una máquina que trabaje con complemento a 2 se tiene una sola representación de 0, en los demás sistemas de representación se tienen 2: +0 y -0.

Resta por Complemento a 2: La sustracción se obtendrá por simple suma del complemento, sin tener en cuenta el arrastre.

$$A = a_1 a_2 a_3 a_4$$

$$B = b_1 b_2 b_3 b_4$$

$$\text{Complemento}(B) = b'_1 b'_2 b'_3 b'_4$$

$$A + \text{Complemento}(B) = c_1 c_2 c_3 c_4 c_5$$

$$\text{Resta}(A-B) = c_2 c_3 c_4 c_5$$

En caso de no producirse arrastre el resultado será negativo. Se deberá complementar el resultado y sumarle 1.

Resta por Complemento a 1: La sustracción se obtendrá por suma del complemento, y luego suma del arrastre producido.

$$A = a_1 a_2 a_3 a_4$$

$$B = b_1 b_2 b_3 b_4$$

$$\text{ComplementoRest}(B) = b'_1 b'_2 b'_3 b'_4$$

$$A + \text{Complemento}(B) = c_1 c_2 c_3 c_4 c_5$$

$$\text{Resta}(A-B) = c_2 c_3 c_4 c_5 + c_1$$

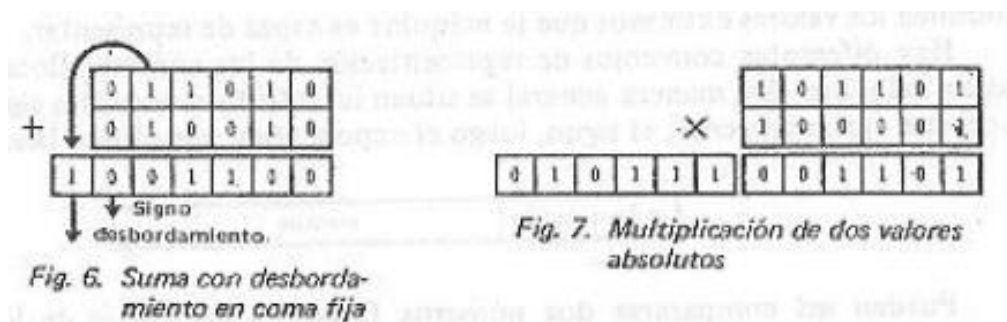
En caso de no producirse arrastre el resultado será negativo. Se debe complementar el resultado.

Concepto de Aritmética de Punto Flotante y Fijo

Formato de Coma Fija: Es la manera más natural de escribir un número binario en una palabra de memoria. Los números se dejan como enteros y depende del programador situar la coma.

El primer bit de cada palabra representa el signo. Por tanto una palabra de n bits puede representar los enteros de $-2^{n-1}+1$ hasta $2^{n-1}-1$ (en el complemento a 2 hasta 2^{n-1}).

La suma de 2 números fijos puede ocasionar desbordamiento. La multiplicación. De 2 números de n cifras da como resultado un número de 2ncifras como máximo.



Formato de Coma Flotante: Su objetivo es calcular con el máximo de precisión. Los números entonces deben encuadrarse de manera de guardar siempre el mayor número de dígitos significativos.

El número se representa de la forma $SM * a^E$

Dónde:

S: es el signo

M: la mantisa. Es la parte de una representación en punto flotante que contiene los dígitos significativos del número a representar. En $2,15 * 10^3$, la mantisa es 2,15.

E: la exponente del número.

a: se escoge generalmente igual a la base del sistema de numeración. En binario $a=2$.

Un ejemplo en el sistema decimal para ver que es una notación de uso muy común:

$$125000000 = 125 \cdot 10^6 = 12,5 \cdot 10^7 = 1,25 \cdot 10^8 = 1250 \cdot 10^5$$

Generalmente se representa de la siguiente manera en la máquina, de información más significativa a menos significativa:

Signo	Exponente	Mantisa
-------	-----------	---------

Signo: un bit, 1 para + y 0 para –

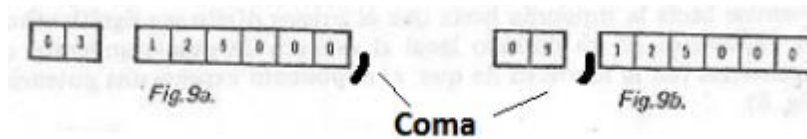
Exponente: Expresa generalmente una potencia de 2.

Si ocupa e bits

Los exponentes negativos varían entre -2^{e-1} y 0. Expresados por valores binarios entre 0 y 2^{3-1} .

Y los positivos entre 0 y $2^{e-1}-1$. Expresados por valores binarios entre 2^{3-1} y 2^e-1 .

Mantisa: Existen 2 convenios, la coma inmediatamente a la derecha (figura 9a) o la coma inmediatamente a la izquierda (figura 9b):



Nótese que el exponente cambia, 03 en el primero y 09 en el segundo.

La normalización de la mantisa en base 2 se consigue desplazando hacia la izquierda la mantisa hasta que el bit a la derecha (figura 9a) o izquierda (figura 9b) del punto de la raíz sea uno.

Ejemplo de Normalización de un número exponencial en base 2.

En el caso de representar los números negativos en complemento a la base o complemento a la base menos 1 el signo queda integrado en la mantisa.

Nota: Si el exponente expresa una potencia de 2 y si los números negativos van representados por complementación, el número está normalizado cuando el bit de signo y el bit de mayor peso de la mantisa tienen valores distintos.

Agrupamientos Binarios

Bit: La codificación binaria posee una calidad BIVALUADA. En el sistema binario de numeración esas condiciones se representan por 0 y 1. Se denomina BIT (contracción de BINARY DIGIT) al dígito binario independientemente de su valor. Es la mínima unidad de información.

Byte: El término BYTE (octeto), se utiliza para describir un conjunto de 8 bits consecutivos que se toman como unidad.

Carácter: Es la cantidad de BITS necesarios para representar los diferentes símbolos del alfabeto (letras, símbolos, signos de puntuación, etc.). La cantidad de Bits utilizados depende del sistema.

Palabra: La palabra es un conjunto de caracteres (de longitud fija o variable dependiendo del caso) que la computadora trata como unidad. Es la unidad de Información procesada por la máquina.

Nota: El byte muchas veces no es práctico para manipular datos en problemas de programación, por lo que se recurre al uso de palabra de acuerdo a la siguiente relación:

$$1\text{Byte}=8\text{Bits}$$

1Palabra=2Bytes
 1DoblePalabra=4Bytes
 1CuadruplePalabra=8Bytes
 1DecaByte=10Bytes

Codificación de la Información Numérica y Alfabética

Codificación de la Información Numérica

¿Cuántos bits se necesitan para representar los 10 símbolos del sistema decimal?

$$l = \log_2 10 = 3,16... < 4$$

Es decir que con 4 bits somos capaces de representar cualquier dígito del sistema decimal. Un código que utilice más de 4 se conoce como CODIGO REDUNDANTE.

Código Ponderado: Se dice que un código es ponderado cuando cada bit tiene un valor diferente dependiendo la posición que ocupe. Ejemplos: BCD (8421), AIKEN (2421), etc.

Decimal Codificado en Binario (BCD por sus siglas en inglés): Cada dígito decimal se codifica con 4 bits y se respeta el “peso” asignado en el sistema binario puro, es decir 8421.

Un ejemplo: $70922_{(10)} = 0111\ 0000\ 1001\ 0010\ 0010$

AIKEN (pesos 2421): como los pesos permiten 2 combinaciones posibles para los números 2, 3, 4, 5, 6 y 7 una de esas combinaciones no se considera.

Construcción de la tabla de código AIKEN

AIKEN				
	2	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	0	1
8	1	1	1	0
9	1	1	1	1

	2	4	2	1
1	1	0	0	0
1	1	0	0	1
1	0	1	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1

No pertenecen a la codificación AIKEN

Exceso de Tres: Cada dígito decimal se representa como BCD excedido en tres. 0010(2) en BCD se vuelve 0101(5) en Exceso de Tres.

Ejemplo: $70922_{(10)} = 1010\ 0011\ 1100\ 0101\ 0101$

Códigos No Ponderados: la representación de cada dígito decimal es en principio arbitraria o responde a características distintas al peso de los bits de acuerdo a su posición. Ejemplo: Código de GRAY.

Código de GRAY: El código de GRAY o “binario reflejado” tiene la peculiaridad que de un decimal al siguiente cambia siempre solo un dígito binario por vez.

Construcción de la tabla del código de GRAY.

I. Se colocan los dos bits (0 y 1) y se traza una línea debajo de ellos (espejo), se copian los bits como si se reflejaran en dicho espejo.

0	
1	
1	
0	

II. se completa la siguiente columna con 1 por debajo del espejo y con 0 por encima del mismo.

0	0
0	1
1	1
1	0

III. Se repite la operación de reflejado con los cuatro números obtenidos.

0	0
0	1
1	1
1	0
1	0
1	1
0	1
0	0

IV. Se completa la tercer columna con 1 debajo del espejo y 0 por encima.

0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	1	1
1	0	1
1	0	0

V. Se vuelve a realizar la operación hasta completar los diez dígitos.

0	0	0	0
0	0	0	1
0	0	1	1
0	0	1	0
0	1	1	0
0	1	1	1
0	1	0	1
0	1	0	0
1	1	0	0
1	1	0	1

→

0	0	0	0	0
1	0	0	0	1
2	0	0	1	1
3	0	0	1	0
4	0	1	1	0
5	0	1	1	1
6	0	1	0	1
7	0	1	0	0
8	1	1	0	0
9	1	1	0	1

Codificación de Información No Numérica

Condiciones para la codificación de caracteres:

- Debe incluir a los números representados en alguna forma (BCD, AIKEN, etc.) y debe poder distinguirse rápidamente números de otros caracteres.

- Debe permitir añadir nuevos caracteres específicos para una aplicación determinada.
- Al transmitir, la representación debe incluir un sistema redundante para detectar errores.

Código EBCDIC (Expanded Binary Code Decimal Interchace Code): Diseñado y utilizado por IBM, sirvió como la base para los códigos normalizados posteriores. Los códigos para los caracteres están representados en una tabla.

No tiene características destacables en cuanto a los caracteres alfabéticos y signos de puntuación, salvo algunas alteraciones en países con distintos alfabetos.

Las operaciones aritméticas se realizan con formato con coma fija.

Usa 8 bits (1byte) para cada carácter. Cada dígito decimal se representa de la siguiente forma.

Zona	Dígito
4 bits	4 bits

La zona ocupa los 4 bits de orden superior del byte, tiene un valor fijo para cualquier número:

$1111_2 = F_{16}$. El dígito representa el valor del número decimal, expresado en BCD.

Los números con zona (desempacados o zoneados) no son aptos para operaciones aritméticas.

Para realizarlas se debe quitar la zona (empaquetar), los números quedan entonces sin zona (empaquetados) para poder trabajarlos.

Al empaquetarlos se les agrega al final medio byte (4 bits) que representan el signo.

Con zona: F3F6F0F4F5

Sin zona positivo: 000036045F

-36045:000036045D

Se utiliza F o C para valores positivos; y B o D para valores negativos.

Dígito ↓	Zona →	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0	0000	NUL	DLE	DS		SP	&	-						{	}	/	0
1	0001	SOH	DC1	SOS						a	j	~		A	J		1
2	0010	STX	DC2	FS	SYN					b	k	s		B	K	S	2
3	0011	ETX	DC3							c	l	t		C	L	T	3
4	0100	PF	RES	BYP	PN					d	m	u		D	M	U	4
5	0101	HT	NL	LF	RS					e	n	v		E	N	V	5
6	0110	LC	BS	EOB	UC					f	o	w		F	O	W	6
7	0111	DEL	IL	PRE	EOT					g	p	x		G	P	X	7
8	1000		CAN							h	q	y		H	Q	Y	8
9	1001	RLF	EM							i	r	z		I	R	Z	9
A	1010	SMM	CC	SM			!		:								
B	1011	VT				.	\$	,	#								
C	1100	FF	IFS		DC4	<	*	%	@								
D	1101	CR	IGS	ENQ	NAK	(	)	_	'								
E	1110	SO	IRS	ACK		+	;	>	=								
F	1111	SI	IUS	BEL	SUB			?	"								

Código ASCII (American Standard Code for Information Interchange): Utiliza 7 bits para representar cada carácter. Una parte de las configuraciones binarias se toma para codificar funciones de control.

Es el más utilizado en la actualidad, pero habitualmente se agrega un octavo bit con la intención de agregar representaciones no previstas y/o utilizarlo como bit de paridad en transmisiones.

Los dígitos también se encuentran separados en zona y dígito.

La zona es siempre 011 y el dígito el correspondiente al número en BCD.

Estos números también deben ser empacados para las operaciones aritméticas.

Digito ↓	Zona →	0	1	2	3	4	5	6	7
		0000	0001	0010	0011	0100	0101	0110	0111
0	0000	NUL	DEL	SP	0	@	P	`	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FT	FS	,	<	L	\	l	
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

Códigos redundantes

Utilizan más bits de los necesarios para representar la información. Los bits extra se utilizan para detectar errores en la transmisión de mensajes.

Códigos Autodetectores: Código que mediante un determinado número de bits redundantes puede detectar si la información es correcta o no.

Control de paridad: No permite detectar errores dobles, pero debido a su simplicidad y a lo baja que es la probabilidad de la ocurrencia de errores en los computadores, es el más utilizado. Consiste en agregar un bit más a los bit de información transmitidos, con el fin de hacer que la cantidad de unos transmitidos sea par (paridad par) o impar (paridad impar).

Ejemplo:

Valor=100101

En paridad par (agregando el bit a la derecha)=1001011

En paridad impar (agregando el bit a la derecha)=1001010

Códigos Autocorrectores: estos códigos permiten determinar si la información recibida es correcta o no y en caso de que no corregirla.

Control 2 en 3: Para transmitir una información cualquiera de n bits, se envían 3 veces esos n bits, de forma sucesiva. Puede ocurrir:

- Que las 3 sean iguales => El mensaje es correcto.
- Las 3 son distintas => La máquina detecta error pero no puede corregirlo.
- 2 son iguales y 1 es distinta => Se considera al distinto incorrecto y se toma el otro como correcto.

Código de Hamming: Permite corregir y detectar uno o más errores producidos durante la transmisión para cualquier número de bits.

Codificación en Hamming para corregir un solo error:

1. Determinar cuántos bits de paridad "p" serán necesarios para el código de "i" bits.

Tendremos $i+p+1$ condiciones que deben identificarse con p bits de control de paridad. Como con p bits podemos formar 2^p combinaciones, entonces la cantidad de p debe ser tal que satisfaga:

$$2^p \geq i+p+1$$

Según esta regla, podemos construir la siguiente tabla:

i	1	2	3	4	5	6	7	8	9	10	11	12		Bits de información
p	2	3	3	3	4	4	4	4	4	4	4	5		Bits de control
n	3	5	6	7	9	10	11	12	13	14	15	17		Bits del mensaje

2. Distribuir los bits de información y los de control. Usaremos la siguiente forma, considerando un mensaje de 4 bits de información:

Posición mensaje	7	6	5	4	3	2	1
	i_3	i_2	i_1	p_2	i_0	p_1	p_0

Los bits de control se colocan en las posiciones que corresponden a las potencias de dos.

Se desea que los dígitos de control que indican el resultado del test de paridad, den (en binario) la posición del dígito erróneo.

3. Determinar el valor de cada bit de control. Se construye una tabla de combinaciones binarias de tantas columnas como la cantidad de bits de paridad determinada por la codificación. Para el ejemplo 3 columnas:

	p_2	p_1	p_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Cada bit de paridad controla la posición que coincide con un 1 en su columna en la tabla binaria. Es decir:

p_0 controla las posiciones: 1, 3, 5 y 7
 p_1 controla las posiciones: 2, 3, 6 y 7
 p_2 controla las posiciones: 4, 5, 6 y 7

El bit de paridad debe hacer que la cantidad de bits con valor 1 que controla sean par.

Control y Corrección:

El receptor del mensaje calcula el valor de los bits de control de la siguiente forma:

$$c_0 = 1 \oplus 3 \oplus 5 \oplus 7$$

$$c_1 = 2 \oplus 3 \oplus 6 \oplus 7$$

$$c_2 = 4 \oplus 5 \oplus 6 \oplus 7$$

NOTA V: los números corresponden a la posición de cada bit utilizado para el cálculo de los bits de control (ej. "3" corresponde al bit ubicado en la posición 3) y el símbolo $\oplus$ designa la operación OR exclusiva.

Si el bit de control es igual a 0 es correcto, si es igual a 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados de forma decreciente es decir: Si $c_2=0$, $c_1=0$ y $c_0=1$ entonces error estará en el bit $100_2=4$ y bastara con cambiar el bit a 1 si es 0 o viceversa.

Unidad 2: Circuitos Digitales

Los ordenadores digitales usan lógica binaria: la corriente pasa o no por los componentes electrónicos del ordenador. La lógica binaria trabaja con variables binarias (si o no, verdadero o falso, 1 o 0, etc.) y operaciones lógicas. Estas características permiten el análisis y simplificación mediante reglas del Algebra de Boole

Algebra de Boole: Boole definió hacia 1850 un algebra aplicable a razonamientos sobre proposiciones lógicas: una proposición solo puede ser falsa(0) o verdadera(1). Shannon, en 1938, aplico este algebra al análisis de circuitos digitales. La corriente pasa o no.

Variable lógica (variable binaria, variable booleana): se denota como A, B, C, etc. Y puede tomar el valor 1(verdadero) o 0(falso). Una variable binaria representa a un bit.

Funciones y Formas Canónicas

Funciones lógicas: Es una función de n variables lógicas que toman valores en el conjunto {0,1}

$$F(A_1, A_2, A_3, \dots, A_n) \in \{0, 1\}$$

Queda perfectamente definida al dar sus valores para cada una de las 2^n posibles combinaciones de las n variables. El cuadro que representa las 2^n combinaciones se llama "tabla de verdad de la función".

Funciones Básicas:

Función constante: $F_1(A)=0 \forall A \in \{0,1\}$ o $F_2(A)=1 \forall A \in \{0,1\}$ Su resultado es siempre el mismo valor.

Función identidad: $F_3(A)=A \forall A \in \{0,1\}$ No altera el valor de A.

Función inversa (complementación, negación): $F_4(A)=\bar{A} \forall A \in \{0,1\}$ Esta función cambia el valor de A por su inverso. A esta función le corresponde el operador de complementación:



Tabla de verdad:

A	F_1	F_2	F_3	F_4
0	0	1	0	1
1	0	1	1	0

Función Intersección (producto lógico): $F(A,B)=A.B$ Satisface la siguiente tabla de verdad:

A	B	$F(A,B)$
0	0	0
0	1	0
1	0	0
1	1	1

Solo es verdadera cuando ambas variables lo son. Le corresponde el operador lógico AND:



Función Reunión (unión o suma lógica): $F(A,B)=A+B$ Satisface la siguiente tabla de verdad:

A	B	$F(A,B)$
0	0	0
0	1	1
1	0	1
1	1	1

Es verdadera cuando cualquiera de las variables lo es. Le corresponde el operador OR:



Función OR Exclusivo: No forma parte de las llamadas “funciones básicas”, al ser una combinación de las mismas, pero definirla es útil luego para la simplificación. $F(A,B)=A \oplus B$ Su tabla de verdad es:

A	B	F(A,B)
0	0	0
0	1	1
1	0	1
1	1	0

Es verdadera cuando solo una(o un número impar) de entradas es verdadera. Su operador lógico:



Teoremas Fundamentales del Algebra de Boole:

Elementos nulos	$A + 1 = 1$ $A \cdot 0 = 0$
Identidad	$A \cdot 1 = A$ $A + 0 = A$
Complemento	$A + \bar{A} = 1$ $A \cdot \bar{A} = 0$
Idempotencia	$A + A = A$ $A \cdot A = A$
Involutiva	$\bar{\bar{A}} = A$
Conmutativa	$A + B = B + A$ $A \cdot B = B \cdot A$
Asociativa	$A + (B + C) = (A + B) + C$ $A \cdot (B \cdot C) = (A \cdot B) \cdot C$
Distributiva	$A + B \cdot C = (A + B) \cdot (A + C)$ $A \cdot (B + C) = A \cdot B + A \cdot C$
Absorción	$A + A \cdot B = A$ $A \cdot (A + B) = A$
Leyes de Morgan	$\overline{A + B} = \bar{A} \cdot \bar{B}$ $\overline{A \cdot B} = \bar{A} + \bar{B}$

Otros Teoremas Útiles:

OR Exclusivo	$\bar{A} \cdot B + A \cdot \bar{B} = A \oplus B$
OR Exclusivo Negado	$A \cdot B + \bar{A} \cdot \bar{B} = \bar{A} \oplus \bar{B}$

Dualidad: Todas las expresiones booleanas permanecen validas si se intercambian los operadores “+” y “.” y los elementos “0” y “1”.

Expresión Equivalente: Una expresión será equivalente a otra si sus tablas de verdad son iguales.

Minitermino: es un producto booleano en el que cada una de las n variables aparece una sola vez (negada o sin negar). Por ejemplo:

Maxitermino: es una suma booleana en el que cada una de las n variables aparece una sola vez (negada o sin negar).

Formas Canónicas (Normales): Una Función lógica que está compuesta por operadores lógicos puede ser expresada en forma canónica usando los conceptos de miniterminos y maxiterminos. Todas las funciones lógicas son expresables en forma canónica, tanto como una "suma de miniterminos" como "producto de maxiterminos".

Generalización de las Formas Canónicas:

Forma Normal Disyuntiva (FND): para hallarla, de las filas donde la función es verdadera, se toma el minitermino correspondiente.

Forma Normal Conjuntiva (FNC): para hallarla, de las filas donde la función es falsa, se toma el maxitermino correspondiente.

Ejemplo:

A	B	C	Maxiterminos	Miniterminos	Salida
0	0	0	$A + B + C$	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	1
0	0	1	$A + B + \bar{C}$	$\bar{A} \cdot \bar{B} \cdot C$	0
0	1	0	$A + \bar{B} + C$	$\bar{A} \cdot B \cdot \bar{C}$	0
0	1	1	$A + \bar{B} + \bar{C}$	$\bar{A} \cdot B \cdot C$	1
1	0	0	$\bar{A} + B + C$	$A \cdot \bar{B} \cdot \bar{C}$	0
1	0	1	$\bar{A} + B + \bar{C}$	$A \cdot \bar{B} \cdot C$	1
1	1	0	$\bar{A} + \bar{B} + C$	$A \cdot B \cdot \bar{C}$	1
1	1	1	$\bar{A} + \bar{B} + \bar{C}$	$A \cdot B \cdot C$	0

$$FND = \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C}$$

$$FNC = (A + B + \bar{C}) \cdot (A + \bar{B} + C) \cdot (\bar{A} + B + C) \cdot (\bar{A} + \bar{B} + \bar{C})$$

Simplificación: que el diseño de un circuito sea lo más simple posible, que cumpla la misma función con la menor cantidad de compuertas posibles, la menor cantidad de entradas a ellas y en dos niveles. Para eso se debe minimizar la función.

Valor Económico de la Simplificación: la simplificación de los circuitos se traduce en un menor número de componentes electrónicos, que "mejoran" la eficacia de su comportamiento al minimizar su complejidad y reducen el número de componentes.

Métodos de Simplificación:

Método Algebraico: consiste en utilizar los teoremas del algebra de Boole hasta llegar a una expresión más simple.

Método Grafico (Tablas o Diagramas de Karnaugh): el diagrama de Karnaugh permite determinar entre expresiones de tipo FND (también expresiones FNC) la más simple, correspondiéndoles circuitos con el menor número de compuertas y/o entradas.

Pasos para la minimización:

1. Representación en una "tabla de Karnaugh" de la función como suma de miniterminos (o producto de maxiterminos).
2. Agrupar todas las celdas positivas, en grupos que tengan una cantidad potencia de 2 de celdas, buscando el menor número de grupos (grupos lo más grandes posibles).
3. Eliminación de variables innecesarias.

Ejemplo de Minimización:

A	B	C	D	Salida
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Tabla de Karnaugh

	AB			
CE	00	01	11	10
00	1	0	0	0
01	1	0	0	1
11	1	1	0	1
10	1	1	0	1

Notar que el orden 11-10 y no 10-11
Se puede agrupar entre extremos (10-00)
como si la tabla se repitiera continuamente.

Función minimizada:

$$\bar{A} + C\bar{A} + E\bar{B} + C\bar{B}$$

Luego de obtener la función minimizada puede operarse con el álgebra de Boole para lograr un circuito más simple. En este caso:

$$\bar{A} + \bar{B}(E + C)$$

Nota: para 5 variables (A, B, C, D y E) lo más sencillo es trabajar con dos diagramas de Karnaugh de 4 variables (B, C, D, E), uno para el valor positivo de una de ellas (A) y otro para su valor negado ($\bar{A}$). Y luego unir las funciones resultantes con una suma y operar con operaciones del álgebra de Boole.

Circuitos Digitales Básicos: Sumadores, Sustractores, Convertidores de Código:

Sumador:

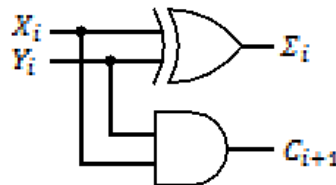
Circuito Combinacional que tiene como entrada 2 números binarios de n bits y da como salida la suma binaria de estos.

Semisumador (medio sumador): Dos entradas, genera resultado y acarreo.

$$C_{i+1} = X_i Y_i$$

$$\Sigma_i = X_i \oplus Y_i$$

X_i	Y_i	C_{i+1}	Σ_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



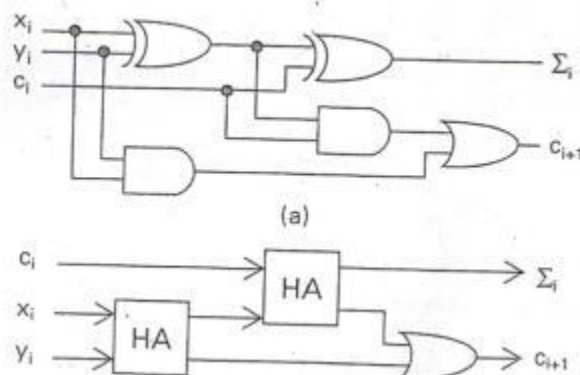
Sumador Completo: Tres entradas (con el acarreo previo), genera resultado y acarreo.

$$C_{i+1} = (Y_i C_i \oplus X_i C_i) + X_i Y_i$$

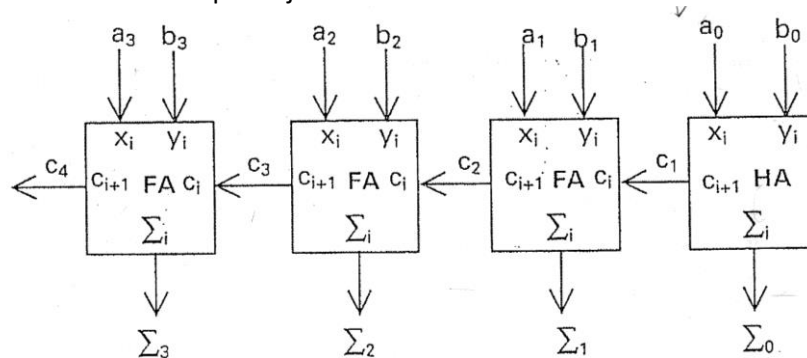
$$\Sigma_i = X_i \oplus Y_i \oplus C_i$$

X_i	Y_i	C_i	C_{i+1}	Σ_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

HA: "Half-Adder" ósea
Medio-Sumador.



Sumador en Serie: Si conectamos un medio sumador seguido de un grupo de sumadores completos uno detrás de otro conseguimos un Sumador en Serie. Cada etapa de sumador debe esperar que la anterior finalice para ejecutarse.



Estos circuitos tienen el problema de que el número de niveles totales del circuito aumenta cuando el número de bits aumenta, lo que hace que el retardo sea mayor a medida que n crece.

Sustractor:

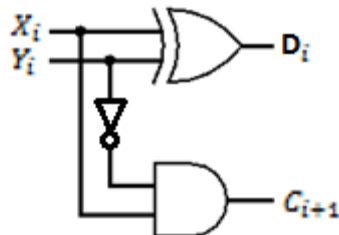
Circuito Combinacional que tiene como entrada 2 números binarios de n bits y da como salida la sustracción binaria de uno por el otro.

Semisustractor (medio sustractor): Dos entradas, genera diferencia y acarreo.

$$C_{i+1} = \bar{X}_i Y_i$$

$$D_i = X_i \oplus Y_i$$

X_i	Y_i	C_{i+1}	D_i
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0



Sustractor Completo: Tres entradas (con el acarreo previo), genera diferencia y acarreo.

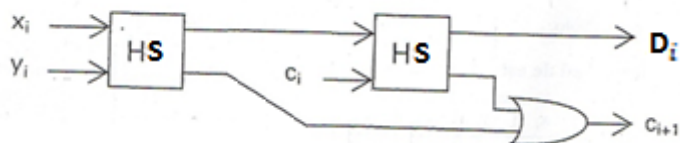
$$C_{i+1} = \bar{X}_i Y_i + \bar{X}_i C_i + Y_i C_i$$

$$\Sigma_i = X_i \oplus Y_i \oplus C_i$$

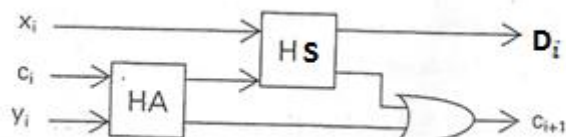
X_i	Y_i	C_i	C_{i+1}	Σ_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Existen 2 formas de realizar un sustractor completo:

1. Se restan sucesivamente X_i y Y_i y después C_i a la diferencia obtenida:

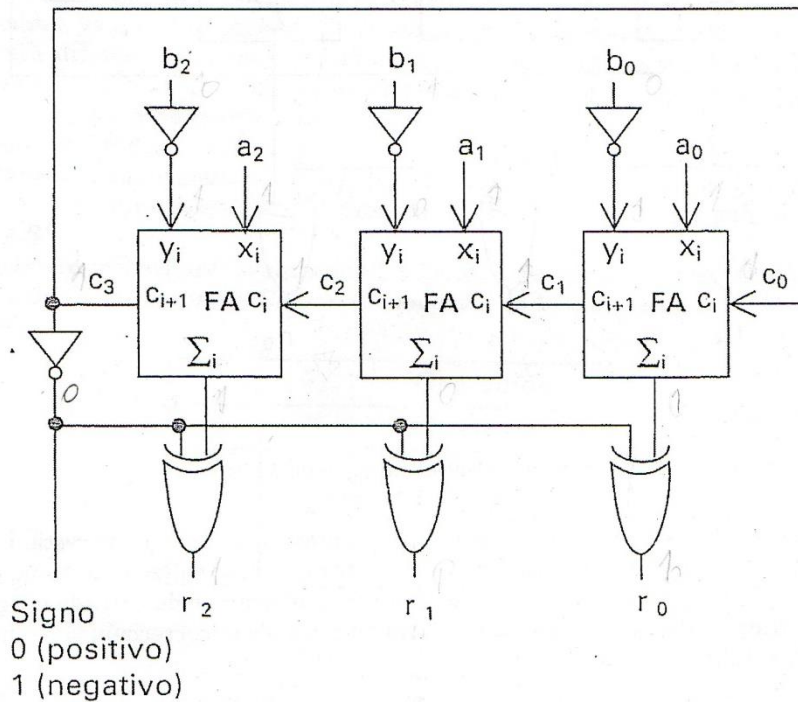


2. Se suman Y_i y C_i y el resultado se le resta a X_i :



HS: "Half-Subtractor" ó sea Medio-Sustractor.

Sustractor en Serio (usando complemento a 1):



(b) restador de 3 bits utilizando complementos a 1.

Convertidores de Código

Las operaciones de codificación, decodificación y transcodificación pueden realizarse físicamente mediante un conjunto de puertas organizadas de forma matricial.

Matriz Rectangular de Decodificación: Se pide decodificar una información binaria de 4 bits, E_1 , E_2 , E_3 y E_4 representativos respectivamente de los pesos 1, 2, 4 y 8 sobre un conjunto de salidas numeradas de 0 a 15.

Realizamos las Ecuaciones:

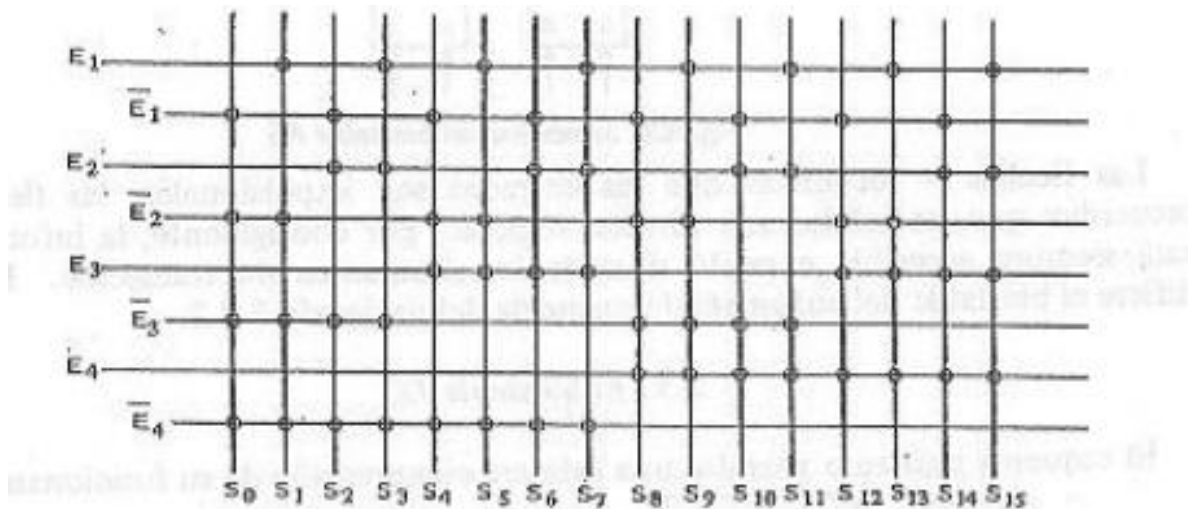
$$S_0 = \bar{E}_1 \bar{E}_2 \bar{E}_3 \bar{E}_4$$

$$S_1 = \bar{E}_1 \bar{E}_2 \bar{E}_3 E_4$$

...

$$S_{15} = E_1 E_2 E_3 E_4$$

Es posible agrupar estos circuitos de forma matricial, cuyas filas son las 8 entradas ciertas o falsas y las columnas las 16 salidas. De la siguiente forma (Los puntos son representaciones de compuertas AND):



Matriz Rectangular de Codificación: se pide codificar 10 cifras decimales E en 4 bits denominados S_1, S_2, S_3 y S_4 de código BCD con pesos 1, 2, 4 y 8 respectivamente.

De la tabla de verdad deducimos las formulas:

E	S_4	S_3	S_2	S_1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

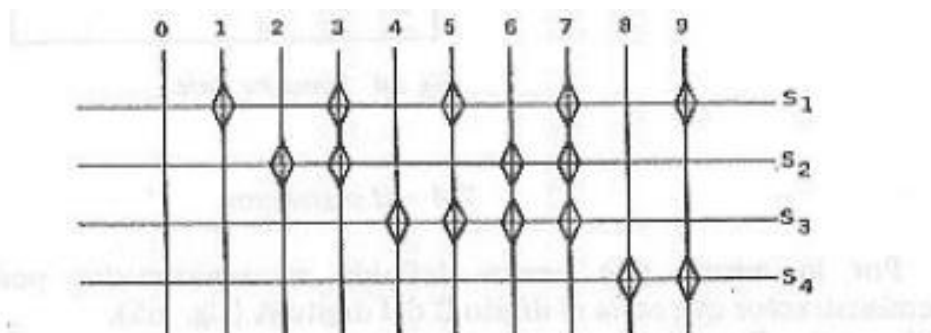
$$S_1 = E_1 + E_3 + E_5 + E_7 + E_9$$

$$S_2 = E_2 + E_3 + E_6 + E_7$$

$$S_3 = E_4 + E_5 + E_6 + E_7$$

$$S_4 = E_8 + E_9$$

Los Circuitos OR necesarios pueden agruparse en una matriz rectangular:



Empleo de Circuitos Combinacionales: Un circuito es Combinacional si trata señales lógicas. Cuyas señales de salida solo dependen de sus entradas.

Circuitos Secuenciales: circuito cuyas señales de salida dependen de las entradas y del tiempo. Un circuito secuencial tiene elementos de retardo o elementos de memoria. Para estudiarlos se trabaja con la Teoría de Autómatas.

Concepto de Autómata Finito: Los autómatas son entes matemáticos, cuyo estudio permite dar un soporte teórico a numerosos desarrollos relacionados con la informática: Teoría de los Circuitos, estudio de lenguajes formales, teoría de algoritmos, etc. Aquí se restringe el estudio del autómata a un ángulo externo tratándolo como una caja negra con entradas y salidas.

Siendo E el conjunto de entradas, S el conjunto de salidas y observando el autómata solo en instantes sucesivos discretos $t, t+1, t+2, \dots$

Entonces: Suponemos que una entrada $E(t)$ en el instante t produce una respuesta $S(t+1)$ en el instante $t+1$. Dicha respuesta depende de $E(t)$ y a su vez también de la función tiempo $H(t)$.

$$S(t + 1) = F[H(t), E(t)]$$

Se puede observar que esta definición es equivalente a la de circuitos secuenciales.

También admitimos 2 hipótesis:

1. La función $H(t)$ está completamente determinada por la historia previa del autómata.
2. El autómata está constituido por un número finito de elementos, y por consiguiente un número finito de elementos de memoria. Y por lo tanto $H(t)$ no puede tomar más que un número finito de valores distintos llamados ESTADOS INTERNOS DEL AUTOMATA.

Funciones de Transición: Si al estado del autómata en el instante t lo denominamos $Q(t)$ entonces la ecuación se convierte en:

$$S(t + 1) = F[Q(t), E(t)]$$

Que expresa que la respuesta a una entrada (estimulo) $E(t)$ no depende más que del estado en el instante t y no de la historia anterior.

Por la misma razón $Q(t+1)$ del autómata en el instante $t+1$ no puede depender más que de su estado anterior $Q(t)$ y de la entrada $E(t)$. Se deduce que:

$$Q(t + 1) = G[Q(t), E(t)]$$

Con estas 2 ecuaciones podemos calcular paso a paso el comportamiento del autómata cualquiera sea la sucesión de estímulos.

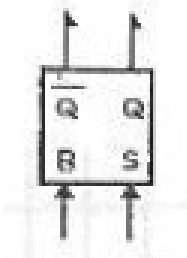
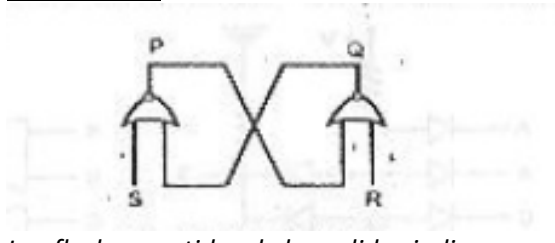
Las funciones F y G se denominan funciones de transición y se representan en forma de tablas o diagramas.

Elementos de memoria: Biestables, Distintos Tipos:

Circuito Biestable (Flip-Flop): es un sistema con dos estados estables. Esta constituido fundamentalmente por 2 circuitos NOT montados en oposición. No puede haber más de dos estados lógicos diferentes que en ausencia de estímulos exteriores, no tienen por qué cambiar, por esta razón este circuito funciona como elemento de memoria.

Tipos de Biestable:

Biestable RS:



Las flechas partidas de las salidas indican que la accesible a menos que el biestable se encuentre en medio de una transición.

informacion esta

Tabla de Verdad, Funcionamiento o Estado

R	S	Q	Q _{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	?
1	1	1	?

$$Q = \overline{\overline{Q} + R}$$

$$\overline{Q} = \overline{Q + S}$$

Tabla de Excitación

Q	Q _{t+1}	R	S
0	0	X	0
0	1	0	1
1	0	1	0
1	1	0	X

En el caso de este Biestable, el estado cambia a 1 al ingresar una señal por la entrada S (set) y cambia a 0 al ingresar una señal por la entrada R (reset). En caso de ingresar señales por ambas entradas el comportamiento del Biestable no puede determinarse.

Biastable JK:

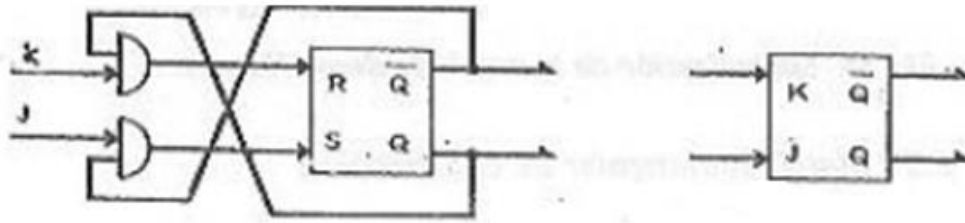


Tabla de Verdad, Funcionamiento o Estado

J	K	Q	Q _{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

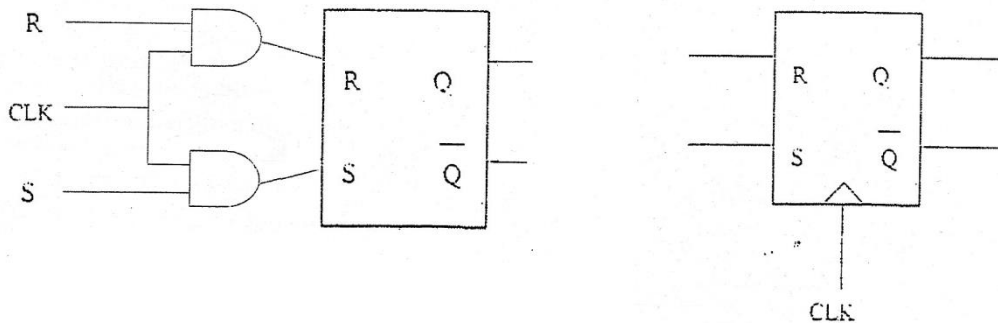
$$Q_{t+1} = J \bar{Q} + \bar{K} Q$$

Tabla de Excitación

Q	Q _{t+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

En el caso de este Biastable, el estado cambia a 1 al ingresar una señal por la entrada J (set) y cambia a 0 al ingresar una señal por la entrada K (reset). En caso de ingresar señales por ambas entradas el estado se complementa.

El Biastable Sincrónico: Con el objeto de lograr una sincronización controlada de los circuitos secuenciales se suele agregar una señal de sincronismo. Con el fin de que los Biastables solo modifiquen sus estados en determinados momentos. En el caso del Biastable RS sincrónico deben agregarse compuertas AND a sus entradas con el fin de solo activarse al recibir dicha señal (CLK):



Biastable D:

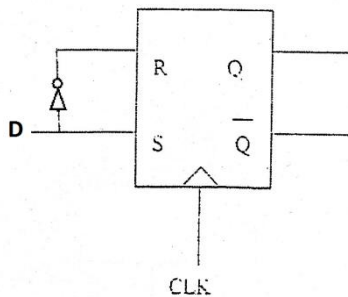


Tabla de Verdad, Funcionamiento o Estado

D	Q	Q _{t+1}
0	0	0
0	1	0
1	0	1
1	1	1

Tabla de Excitación

Q	Q _{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

En el caso de este Biestable (que debe ser sincrónico, es decir, esperar una señal de sincronismo) el estado cambia a 1 al ingresar una señal por la entrada D (set) y cambia a 0 al no ingresar por dicha entrada (ingresar 0).

Biestable T:

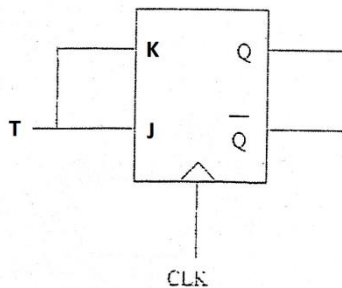


Tabla de Verdad, Funcionamiento o Estado

T	Q	Q _{t+1}
0	0	0
0	1	1
1	0	1
1	1	0

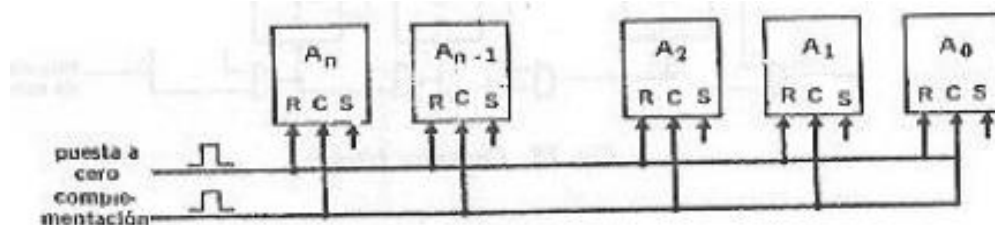
Tabla de Excitación

Q	Q _{t+1}	D
0	0	0
0	1	1
1	0	1
1	1	0

En el caso de este Biestable (que debe ser sincrónico, es decir, esperar una señal de sincronismo) el estado se complementa al recibir una señal por su entrada T. En caso de no recibir señal el estado se mantiene.

Registros: el Biestable permite almacenar una información binaria elemental. El registro compuesto por varios Biestables, permite almacenar una información de varios dígitos.

Operaciones elementales sobre registros: un registro puede ponerse a cero o complementarse al enviar una señal por la entrada adecuada:



Carga y Transferencia: Puede transferirse información de un registro a otro. En la imagen se activa la señal SRF que transfiere la información del registro F al Bus (explicado más adelante) y luego la señal ENA y ENB que cargan el registro A y B respectivamente con dicha información.

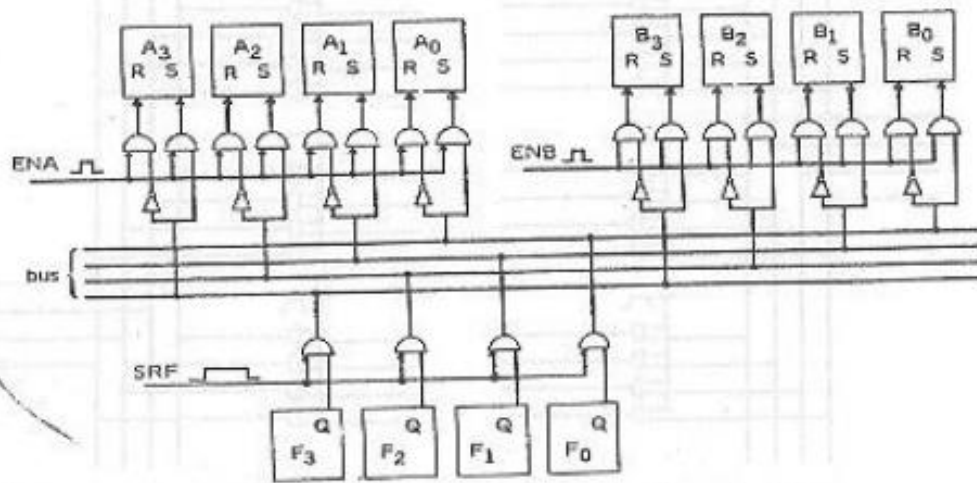
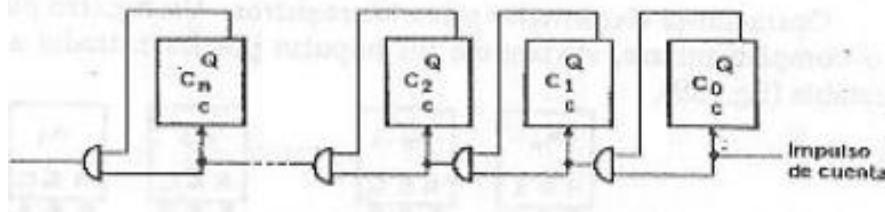


Fig. 71. Transferencia por medio de un bus

Contadores:

Los contadores de impulsos son registros cuyo contenido se incrementa en 1 cada vez que se presenta un impulso a la entrada de cuenta. Son capaces de contar el número de impulsos recibidos. Como ejemplo, el contador binario:



(Los registros en la imagen tienen entradas de complementación, a fines prácticos puede decirse que son Biestables T).

El impulso (t) complementa el valor del primer registro (menor peso) y en el caso de ser 1 el valor anterior (t-1) propaga el arrastre al siguiente registro y así sucesivamente.

Registros de Desplazamiento: con el cableado correspondiente se consigue desplazar el contenido de un registro una determinada cantidad de lugares ya sea a derecha o a izquierda al recibir un impulso.

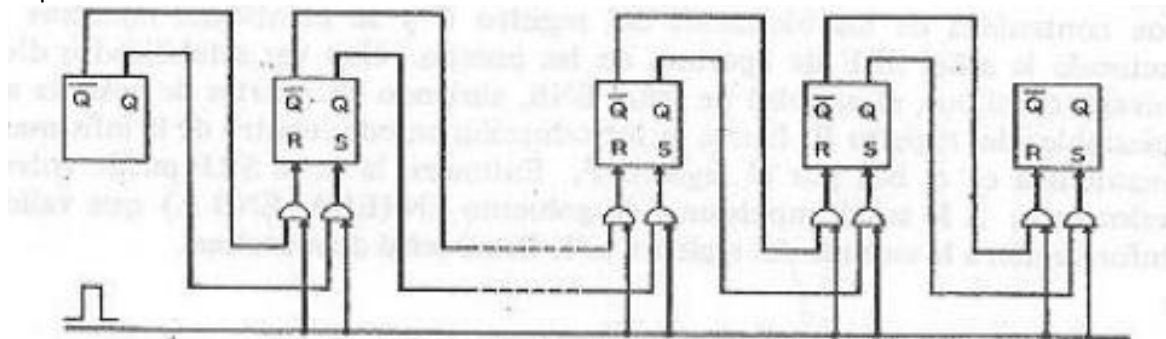


Fig. 74. Registro de desplazamientos a derecha

Registro Acumulador:

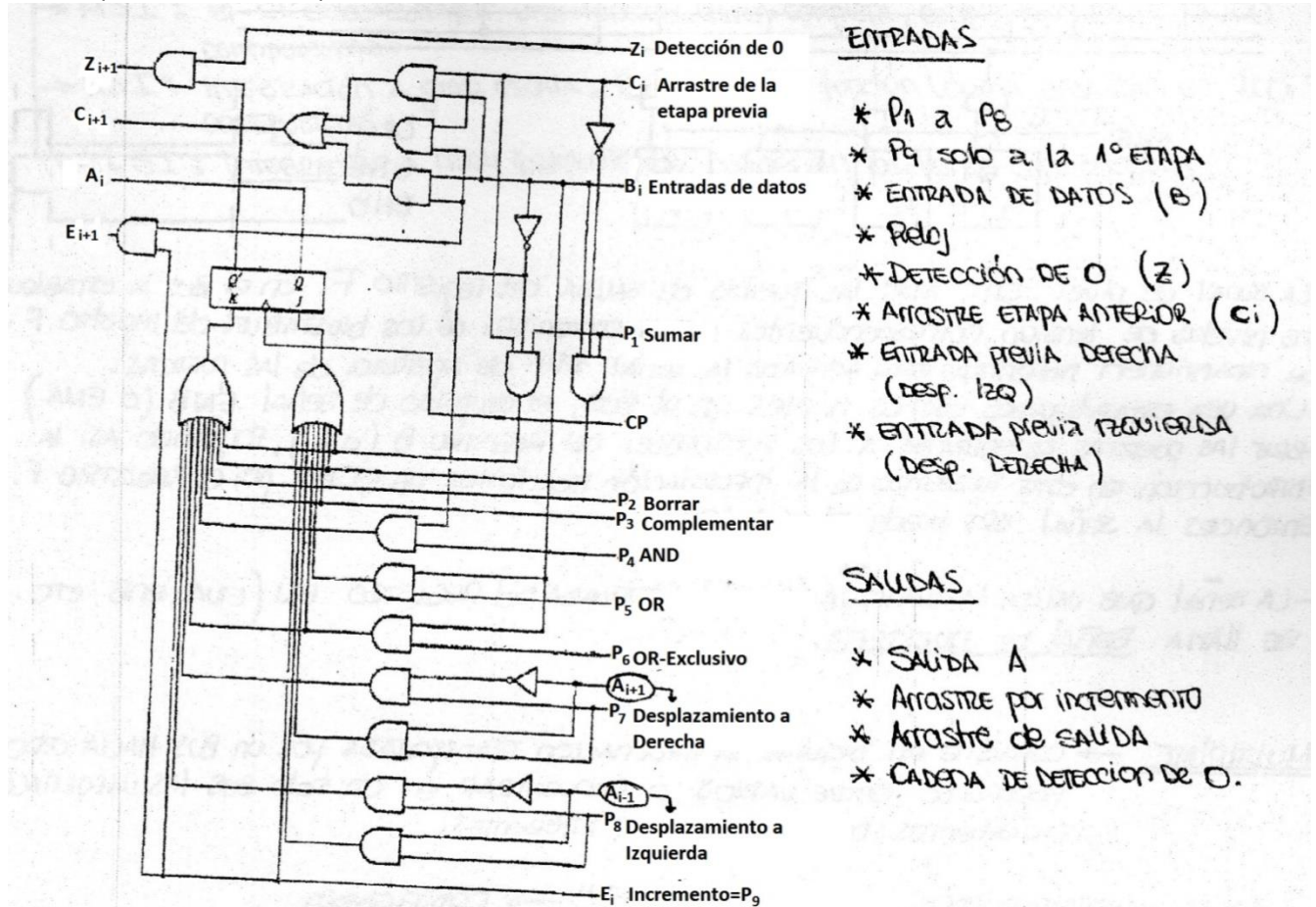
Es un registro multipropósito que opera con datos previamente almacenados en memoria. Se utiliza para ejecutar la mayoría de las instrucciones. El registro acumulador trabaja en conjunto

con la ALU (Unidad Aritmético-Lógica) que explicaremos después.

Se utiliza para realizar suma, resta, intercambio de posiciones de memoria y varias otras instrucciones. Las operaciones incluidas son las que deben incluirse en el procesador particular. Una etapa de acumulador suele incluir todas las microoperaciones que fuimos describiendo individualmente.

El acumulador cuenta con una entrada de datos y además entradas para determinar la operación a realizar.

Una etapa de acumulador podría verse así:



El Multiplaje: consiste en desviar la información transportada por un bus hacia otro escogido entre varios o en concentrar en un solo bus las informaciones procedentes de varios buses diferentes.

Tipos de Circuitos Integrados: Atendiendo al nivel de integración -número de componentes- los circuitos integrados se pueden clasificar en:

- TTL: Lógica de transistores (amplia cantidad de funciones digitales).
- ECL: Lógica de acoplamiento de emisor (operaciones de alta velocidad).
- MOS: Semiconductor de óxido de metal (alta densidad de componentes).
- CMOS: Semiconductor de óxido de metal complementario (bajo consumo energético).
- I²L: Lógica de inyección integrada (alta densidad de componentes).

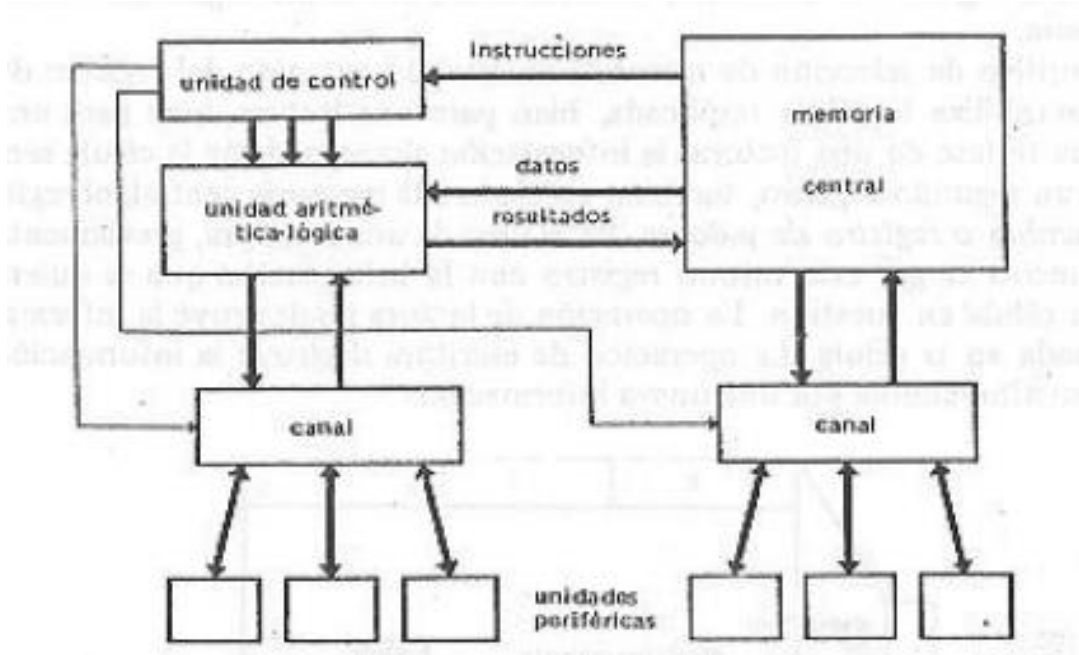
Unidad 3: Arquitectura de los Procesadores

Descripción e Interrelación de los Componentes de un Computador:

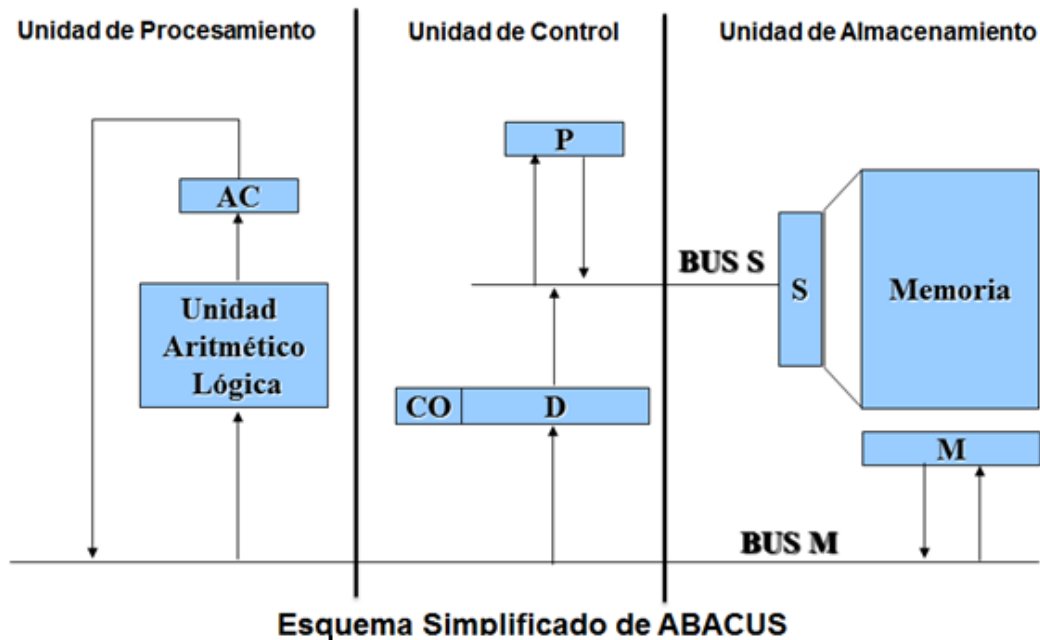
La mayoría de los computadores funcionan de acuerdo a la Arquitectura Von Neumann. Es Von Neumann quien hacia el 1945 promueve el paso decisivo hacia la mecanización del tratamiento digital de información, con la invención de 2 conceptos:

- El programa registrado: Von Neumann tuvo la idea de usar las memorias del computador para almacenar el programa para poder así ejecutarlo cuando fuera necesario.
- La ruptura de secuencia: Von Neumann fue el primero en dotar ha la maquina con lo que conocemos como “salto condicional”.

Arquitectura Von Neumann:



Un ejemplo de Arquitectura Von Neumann son los Ordenadores tipo ABACUS:



S= Registro de Selección de Memoria.

P= Contador Ordinal de Programa.

M= Registro de Intercambio o Palabra de N bits.

CO + D= Registro de instrucción. CO= Código de Operación P bits. D= Dirección M bits.

AC= Acumulador de N bits.

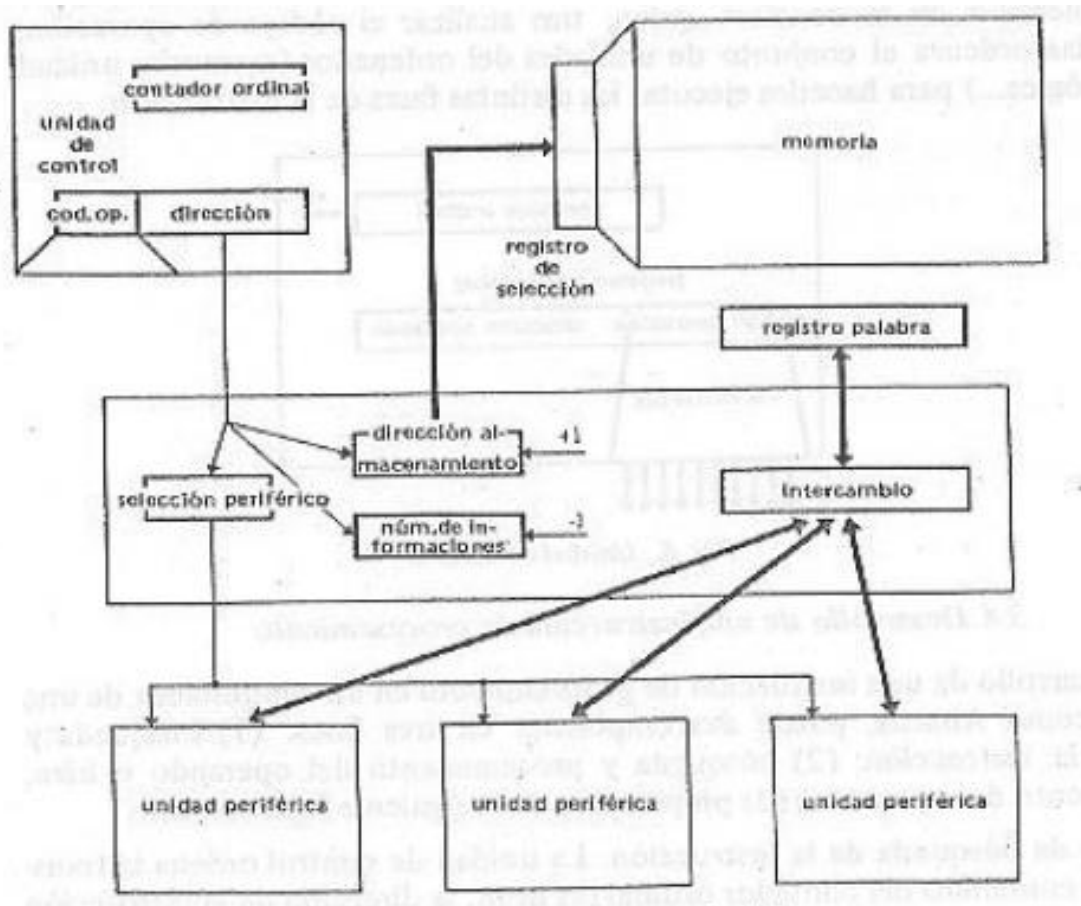
Unidad Central de Proceso: Es el conjunto de la unidad de Control y la Unidad Aritmético Lógica.

Unidad de Control: es la sección encargada de extraer las instrucciones de MC, analizarla y establecer las conexiones necesarias dentro de la ALU para su ejecución.

Memoria Central: El programa es registrado en Memoria antes de comenzar su ejecución. Esta memoria, gracias a la cual puede ejecutarse el programa, se llama memoria central o bien memoria principal y en torno suyo se organiza el resto de diferentes unidades de la máquina. La memoria central almacena 2 clases de información: instrucciones de programa que la maquina debe ejecutar, y datos (operandos) con los cuales efectuara la maquina el tratamiento de dichas instrucciones.

Puede considerarse un conjunto de células, cada una con la posibilidad de almacenar una información (una palabra diremos a menudo): dato o instrucción. Las células están numeradas y la unidad de control conoce cada célula por su número, llamado dirección. La unidad de control puede pedir leer el contenido de una célula o escribir una nueva información en ella.

Canal:



Es el encargado en general de gestionar la transferencia en ambos sentidos de un conjunto de informaciones entre la MC y los periféricos.

Para inicializar la transferencia instrucciones especiales de entrada salida deben suministrar al canal, por una parte la dirección de la unidad periférica implicada y por otra la dirección para el almacenamiento de la primera información y el número de informaciones a transferir, En lo sucesivo el canal se ocupara completamente de la transferencia, por cada información transferida añadirá 1 a la dirección de almacenamiento y restara 1 al número de informaciones a transferir. Luego avisara a la UC cuando todas las informaciones hayan sido transferidas.

Unidades de Entrada y de Salida: Existen 2 grandes clases de unidades periféricas, las unidades de comunicación (impresoras, lectora de tarjetas, etc.) y las memorias auxiliares (discos, cintas magnéticas, etc.). Las unidades periféricas se conectan a través de los canales mediante los registros de intercambio y Selección de periférico. La unidad de control es la encargada de administrar estos intercambios.

Memorias: Dispositivo electrónico capaz de almacenar información, de modo que el órgano que se sirva ella pueda obtenerla en cualquier instante.

Punto de memoria: soporte físico de un bit.

Diversos tipos de fenómenos físicos pueden usarse para la generación de un punto de memoria.

❖ FF con semiconductores (Usada en la Memoria Central)

- SRAM
- DRAM

❖ Superficies magnéticas

- Discos magnéticos
- Cintas magnéticas

❖ Superficie reflectante:

- CD-ROM
- Disco óptico borrable

❖ Discos magneto-ópticos

Jerarquía de Memoria:



- Registros del procesador: forman parte de la CPU. Integrada en el microprocesador. Es un conjunto de Biestables. Son la forma más rápida del sistema de almacenar datos.
- Memoria cache: es un conjunto de datos duplicados de otros originales, cuando los originales son costosos de acceder. Cuando se accede por primera vez a un dato se lo copia a la cache,

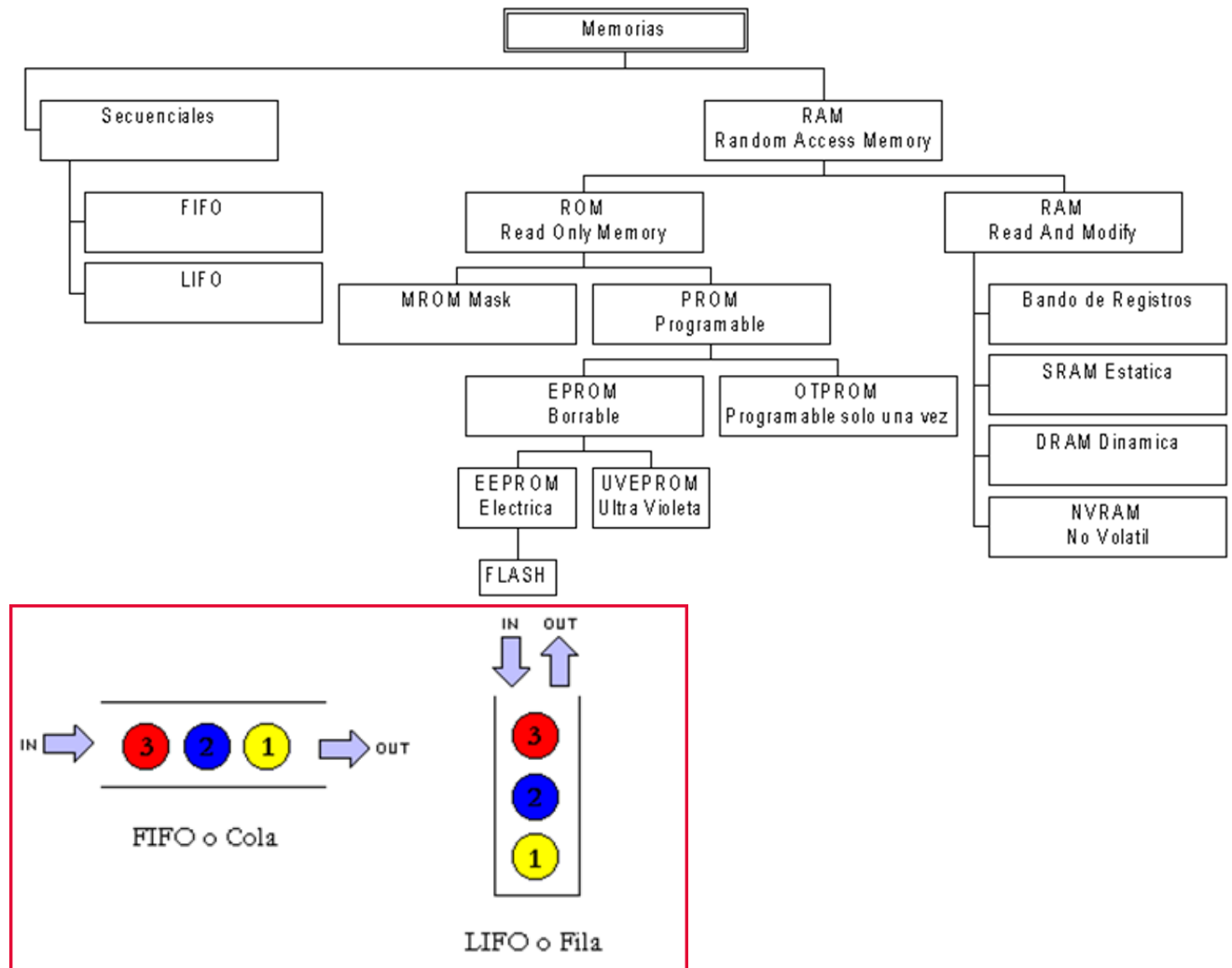
los siguientes accesos se realizan a dicha copia, acelerando el proceso. Su función es mejorar el rendimiento del procesador central sin aumentar el costo del sistema. Se sitúa entre CPU y MC. Se trabaja con 2 niveles de cache: nivel 1, cache interno del microprocesador tan rápido como los registros; y nivel 2, cache externo del procesador, conformado por chips SRAM. Cada vez que se ordena leer la MC, el cache interno provee la información si la posee, si no se recurre al cache externo que la copia al interno, si no se encuentra en el externo se pide a MC.

- Memoria central o principal: En este nivel residen los programas y los datos para su ejecución. Está dividida en celdas identificadas por una dirección. La información contenida es accedida mediante el bus de dirección.
- Memoria Secundaria (discos duros): dispositivos no volátiles que emplean un sistema de grabación magnética digital. La información se pasa del disco a la memoria central para ser utilizada.
- Almacenamiento fuera de línea: Memorias extraíbles de bajo costo. Por ejemplo cintas magnéticas, cd's, etc. Se manejan como periféricos.

Características:

- Volatilidad: se dice que una memoria es volátil si corre el riesgo de ver alterada su información por un fallo de alimentación eléctrica
- Lectura y Escritura: son las operaciones básicas de las memorias. Se realiza escritura cuando se registran informaciones en memoria y lectura cuando se extraen informaciones previamente registradas. La lectura puede ser destructiva (la información se borra al ser leída) o no destructiva. La escritura puede exigir o no un borrado previo.
- Direccionamiento: para localizar una información dentro de la memoria, la computadora debe poder ubicar donde fue colocada anteriormente. Esta es la función de direccionamiento.
- Modo de Acceso: es el método utilizado para acceder a la información. Puede ser:
 - Aleatorio, Directo, Selectivo: Se tiene acceso a cualquier información cuya dirección sea conocida, en cualquier momento en el mismo intervalo de tiempo.
 - Secuencial: significa que acceder a una unidad de información tomará un intervalo de tiempo variable, dependiendo de la ubicación de la información. Un ejemplo son las cintas magnéticas donde la cinta debe desenrollarse para acceder a la información siguiente.
 - Mixto: caso de los discos magnéticos. Es directo en cuanto a la selección de una pista. Y luego secuencial dentro de dicha pista.
- Tiempo de Acceso: Se divide en lectura, escritura y reescritura:
 - Lectura: es el tiempo que transcurre entre el instante en que se lanza la operación de lectura y el instante en el que se dispone de la primera información buscada.
 - Escritura: es el tiempo que transcurre entre el instante en que se la información a guardar y su dirección, y el instante en el que se almacena realmente la información.
 - Reescritura: Tiempo que toma realizar la operación de escritura cuando ya existe información en la dirección donde se planea guardar la nueva información.
- Caudal: número máximo de informaciones leídas o escritas en la unidad de tiempo.
- Capacidad: Cantidad de palabras que la memoria puede almacenar. Se expresan habitualmente en potencias de 2: 1024, 4096, 8192 palabras, etc.

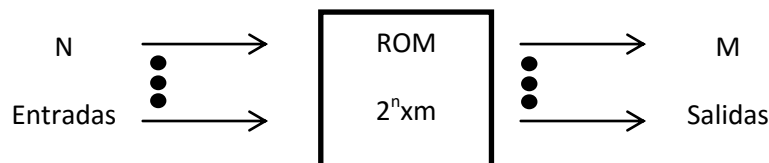
Clasificación:



Funcionamiento:

Tipo de memoria	Tiempo de acceso	Costo x Mb	Tamaño Típico
Registros	1ns	Alto	1kb
Cache	5-20ns	\$100	1Mb
Mem. Principal	60-80ns	\$1,10	64Mb
Disco Duro	>100ns	0,05	4Gb

Memorias ROM (memorias de solo lectura):



Las memorias ROM no son volátiles.

Consiste en n líneas de entrada y m líneas de salida. Cada combinación de bits de entrada es una dirección $\Rightarrow n$ entradas $\rightarrow 2^n$ direcciones. Cada combinación de bits de salida es una palabra $\Rightarrow$ el número de bits por palabra es igual al número de líneas de salida m .

ROM 32x8 -> ROM de 32 palabras de 8bits. $2^5=32$ -> 5 entradas y 8 salidas

- ROM (ReadOnlyMemory): se graban una vez por el fabricante.
- PROM (ROM programable): se graban una vez por el usuario.
- EPROM (Erasable-Programmable ROM): se graban varias veces por el usuario, el borrado se realiza con luz ultravioleta.
- EEPROM (ElectricallyErasableProgrammable ROM): se graban varias veces por el usuario, el borrado se realiza eléctricamente posición a posición.
- Memorias FLASH (EEPROM rápidas): se graban varias veces por el usuario, el borrado se realiza eléctricamente de una sola vez

En una pc la porción de memoria que guarda los programas que se ejecutan para permitir el correcto encendido del sistema y la carga del sistema es ROM y se denomina BIOS.

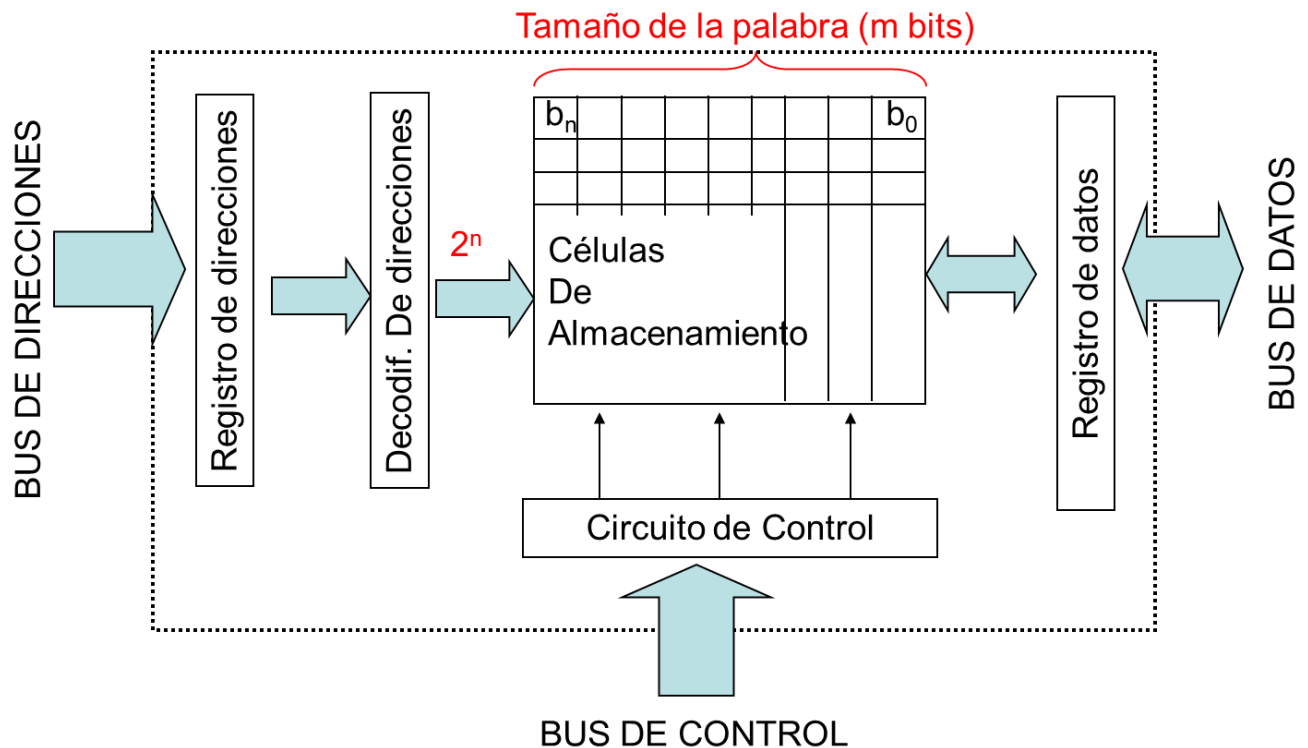
Memoria RAM (Memoria de Lectura y Escritura, no confundir con RAM, memoria de acceso aleatorio):

Las memorias RAM son volátiles, es decir que si se interrumpe la energía se pierden los datos y solo pueden utilizarse como almacenamiento temporal.

Existen dos tipos de RAM:

- SRAM (Static RAM): Se fabrica con Biestables. Son capaces de mantener su información mientras tengan alimentación eléctrica, sin necesidad de refresco. Lectura no destructiva, mayor costo, mayor velocidad, menor consumo, menor capacidad, utilizada para accesos más rápidos.
- DRAM (Dinamic RAM): Se fabrican con condensadores. Necesita ciclos de refresco para mantener sus datos. Lectura destructiva. Generalmente usada como memoria principal.

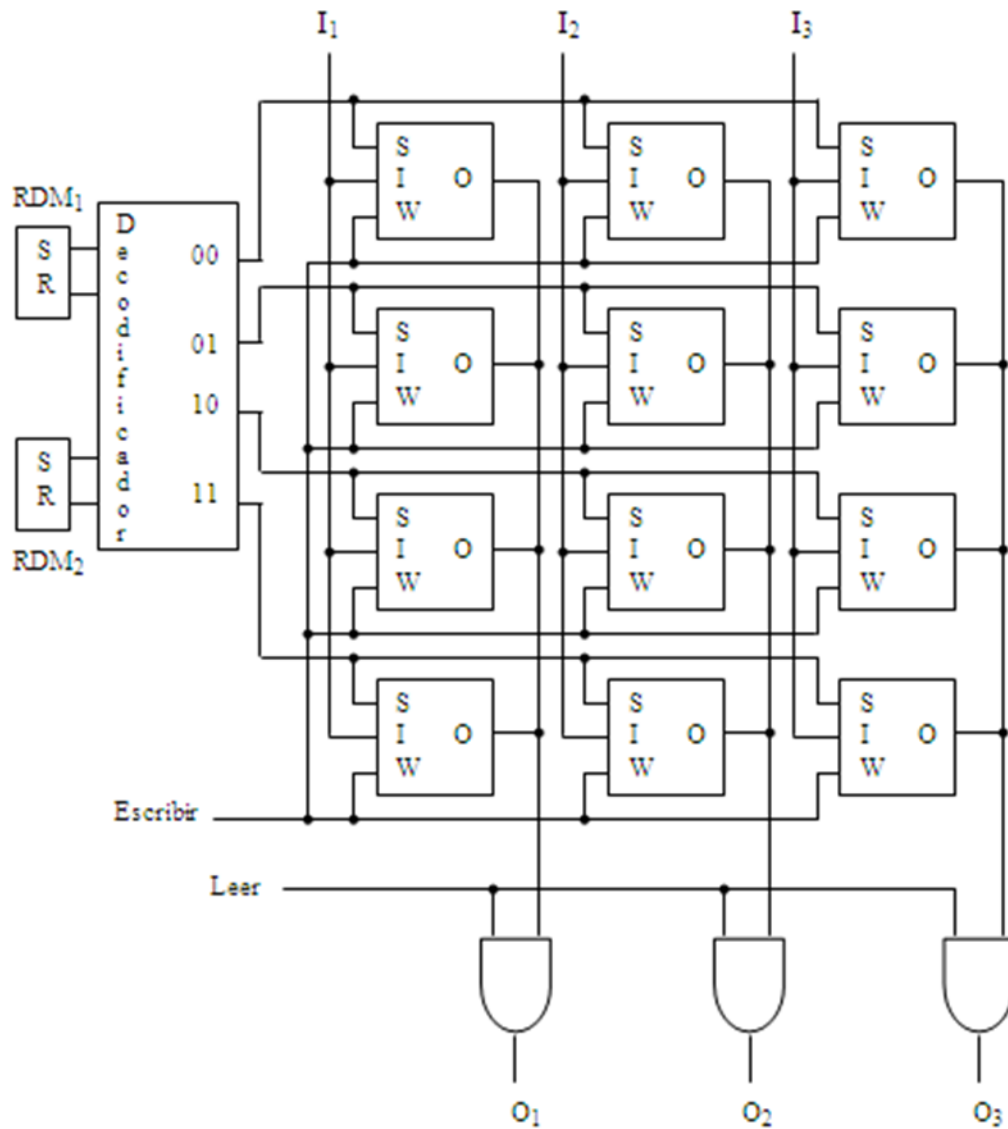
SRAM: Diagrama de bloque:



Las memorias SRAM usan Organización 2D de memoria (Memoria de Selección Lineal).

Organización General de Memoria (memoria de 4 palabras de 3bits)

Datos de entrada

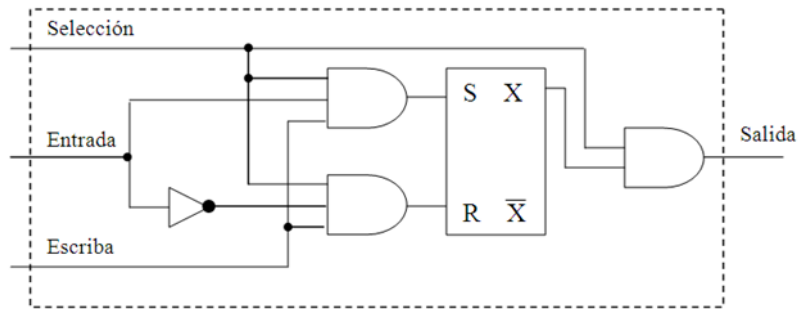


Consiste en una matriz de Biestables, cada uno atravesado por 2 hilos perpendiculares, un hilo de palabra (selección) y un hilo de bit (entrada). Los Biestables de la misma fila memorizan una palabra.

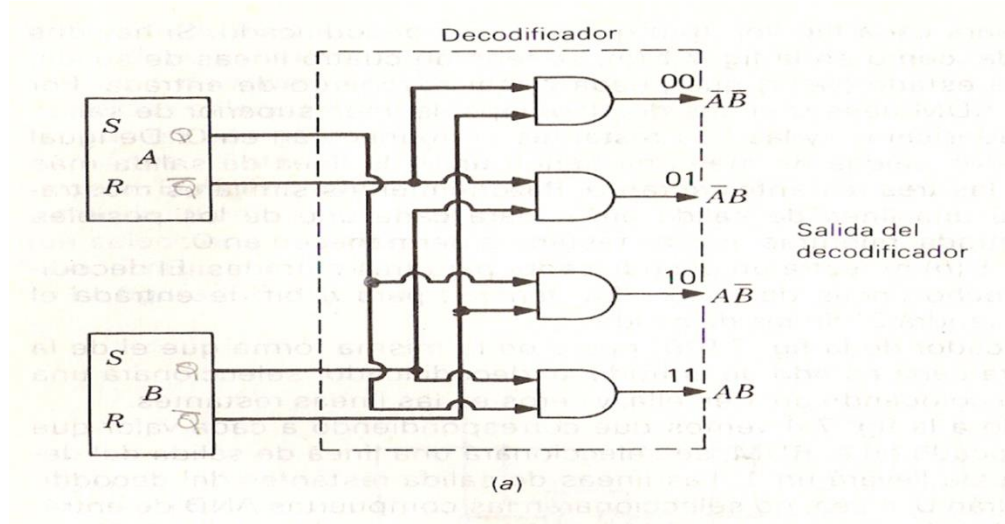
Para la Escritura: Se envía una señal por el hilo de palabra y otra por el hilo de bit para todos los bits a los que les corresponda un 1 en su configuración binaria. Además se habilita la señal de escritura. Únicamente los Biestables atravesados por ambas señales se verán afectados. La palabra que va escribirse se carga en el RPM (Registro de Palabra de Memoria), que pasara la información a las líneas $I_1, I_2, \dots, I_n$ y en el RS (Registro de Selección) se carga la dirección, que para al decodificador.

Para la Lectura: Se envía una señal por el hilo de palabra y al estar cargado con un 1 el Biestable habilita la salida por la compuerta AND. Además se activa la señal de "Leer". Lo que habilita la salida por $O_1, O_2, \dots, O_n$

Punto de Memoria:



Dispositivo de Selección (Decodificador):



Formulas (no se consideran las compuertas dentro de los Biestables):

Numero de palabras= x

Longitud de palabra= m

$N \geq \log_2 x$

Decodificador:

AND= x

Entradas= $x \cdot n$

NOT= Entradas/2

Puntos de Memoria (Memoria Central):

Punto de Memoria= $x \cdot m = PM$

AND= $PM \cdot 3$

Entradas= $PM \cdot 8$

NOT= PM

Lectura:

AND= m

Entradas= $m \cdot 2$

OR= m

Entradas a los OR= PM

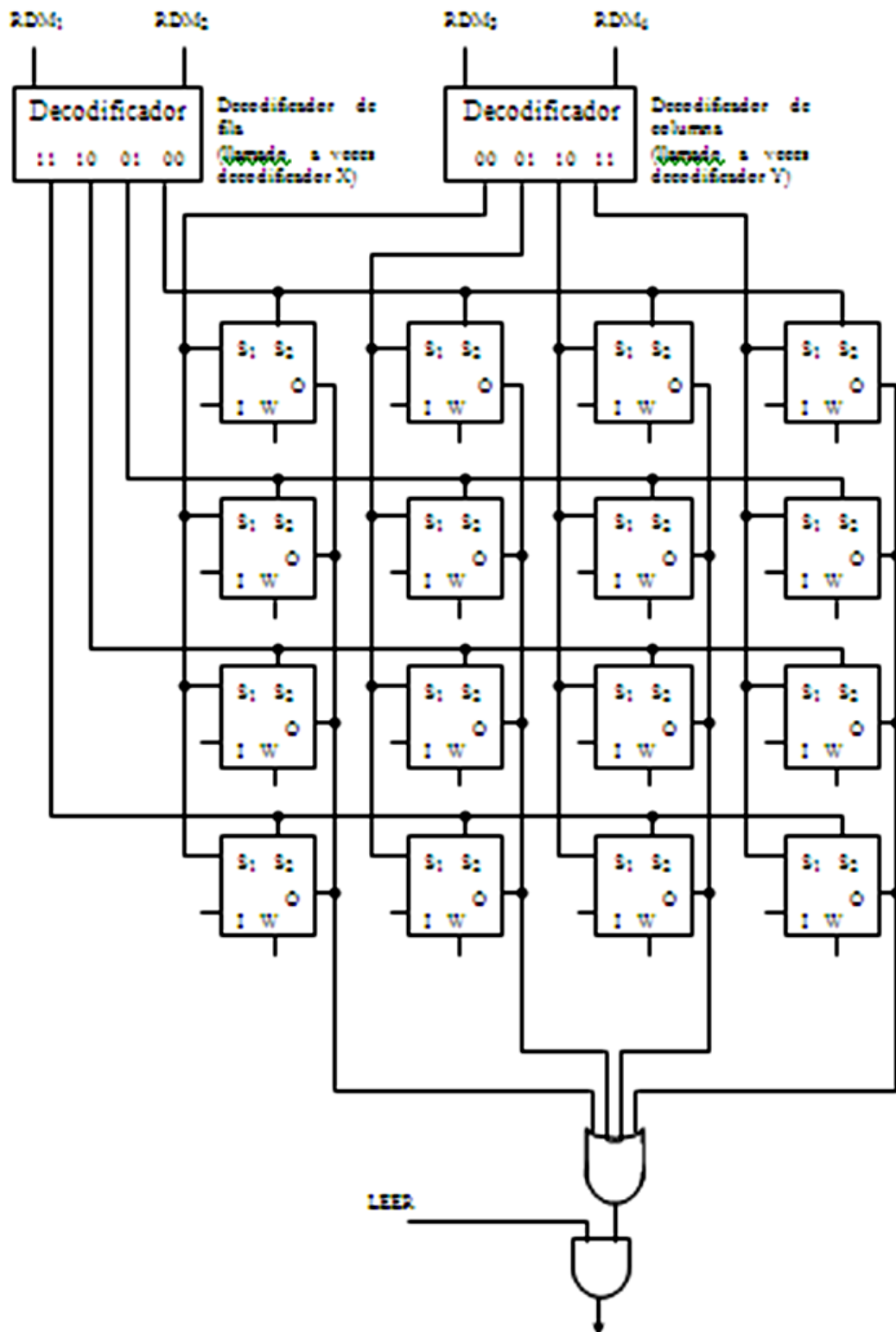
Escritura:

AND=0

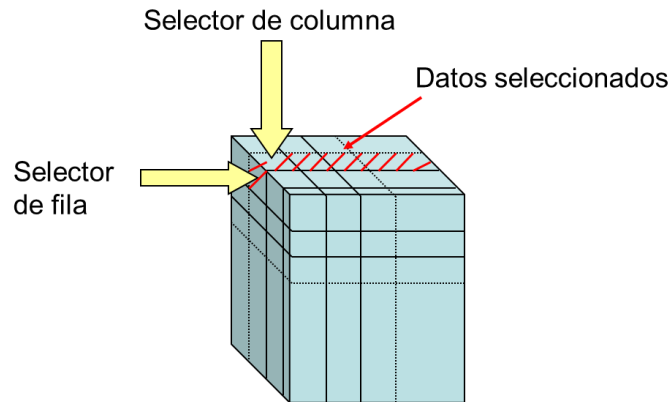
Entradas=0

DRAM: Las memorias DRAM usan Organización 3D de memoria (Memoria de Selección por Corrientes Coincidentes).

Organización General de Memoria



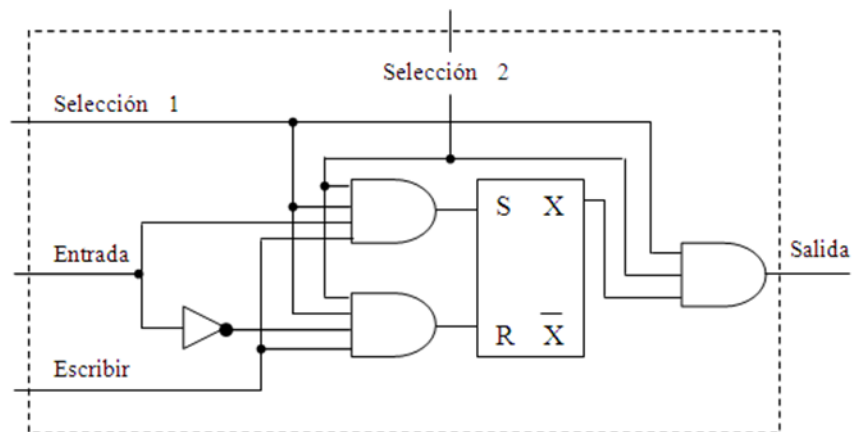
Parte de la decodificación la realiza la propia organización de memoria. Esta se divide en m matrices 2^n Biestables, donde m es el número de bits de la palabra de memoria y n es el número de bits del RS. Cada Biestable de la i -ésima matriz corresponde al bit de peso i de una las 2^n palabras de memoria. Cada Biestable es alcanzado por 4 hilos (2 para la selección de filas y columnas, 1 para la entrada de información y otro para la habilitación de escritura). Se utilizan 2 decodificadores.



Para la Escritura: Se envía una señal por el hilo de fila, otra por el hilo de columna, y otra por el hilo de entrada para todos los bits a los que les corresponda un 1 en su configuración binaria. Además se habilita la señal de escritura. Únicamente los Biestables atravesados por las 3 señales se verán afectados. La palabra que va escribirse se carga en el RPM (Registro de Palabra de Memoria), que pasara la información a las líneas $I_1, I_2, \dots, I_n$ y en el RS (Registro de Selección) se carga la dirección, que para al decodificador.

Para la Lectura: Se envía una señal por el hilo de fila y también por el de columna y al estar cargado con un 1 el Biestable habilita la salida por la compuerta AND. Además se activa la señal de "Leer". Lo que habilita la salida por $O_1, O_2, \dots, O_n$

Punto de Memoria:



Formulas (no se consideran las compuertas dentro de los Biestables):

Numero de palabras= x

Longitud de palabra= m

$N \geq \log_2 x$

$P+Q=N$

Decodificador:

$AND=2^p+2^q$

$$\text{Entradas} = 2^p \cdot p + 2^q \cdot q$$

$$\text{NOT} = \text{Entradas} / 2$$

Puntos de Memoria (Memoria Central):

$$\text{Punto de Memoria} = x \cdot m = \text{PM}$$

$$\text{AND} = \text{PM} \cdot 3$$

$$\text{Entradas} = \text{PM} \cdot 11$$

$$\text{NOT} = \text{PM}$$

Lectura:

$$\text{AND} = m$$

$$\text{Entradas} = m \cdot 2$$

$$\text{OR} = m$$

$$\text{Entradas a los OR} = \text{PM}$$

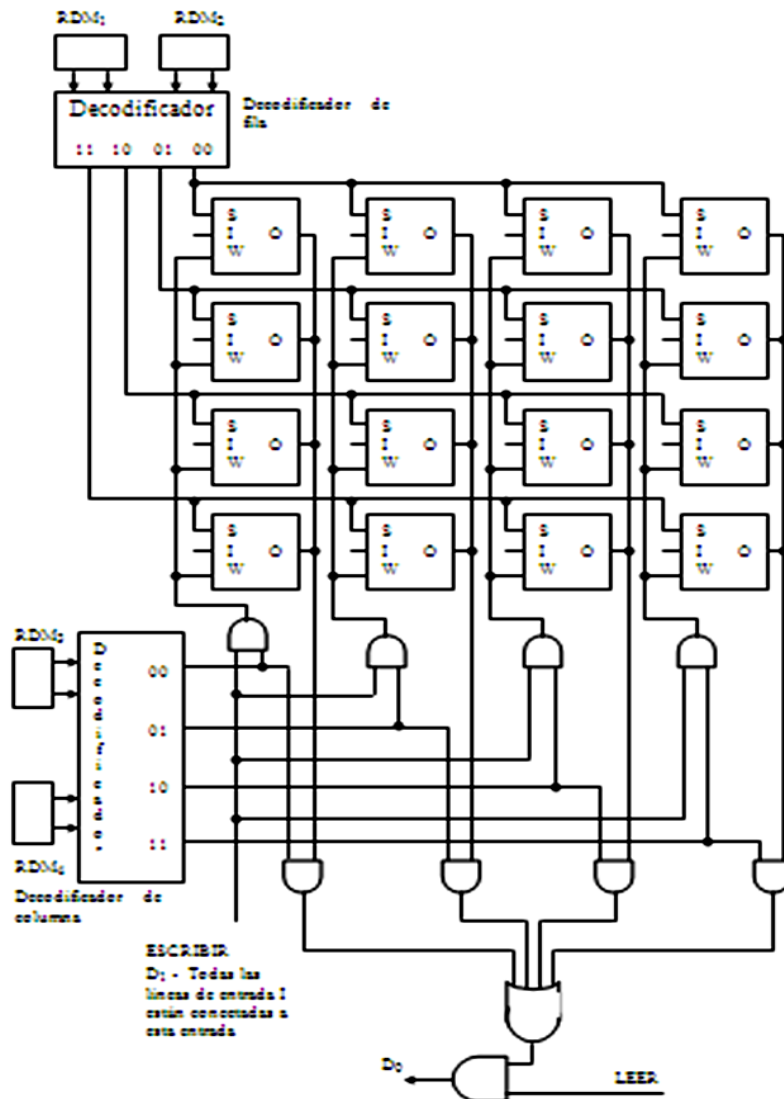
Escritura:

$$\text{AND} = 0$$

$$\text{Entradas} = 0$$

Memorias 2 y medio D:

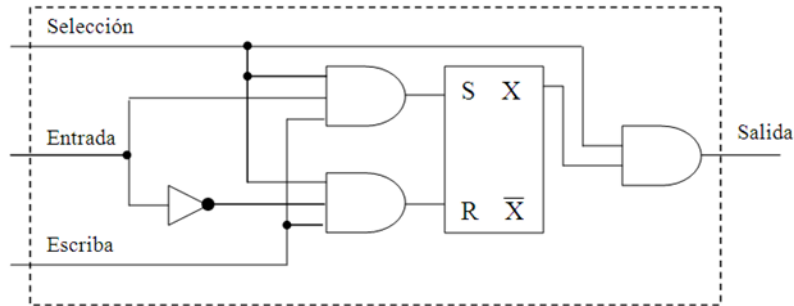
Organización General de Memoria



Estas memorias se caracterizan por tener celdas 2D y Organización 3D (usa 2 decodificadores). El esquema de selección usa un control de las señales de Leer y Escribir para así lograr una organización 3D

El primer decodificador selecciona la fila y el otro combina (mediante un AND) con la señal de lectura/escritura para seleccionar la columna.

Punto de Memoria:



Formulas (no se consideran las compuertas dentro de los Biestables):

Numero de palabras= x

Longitud de palabra= m

$N \geq \log_2 x$

$P+Q=N$

Decodificador:

$AND = 2^p + 2^q$

Entradas = $2^p \cdot p + 2^q \cdot q$

NOT = Entradas/2

Puntos de Memoria (Memoria Central):

Punto de Memoria = $x \cdot m = PM$

$AND = PM \cdot 3$

Entradas = $PM \cdot 8$

Lectura:

$AND = (2^q + 1) \cdot m$

Entradas = $AND \cdot 2$

Escritura:

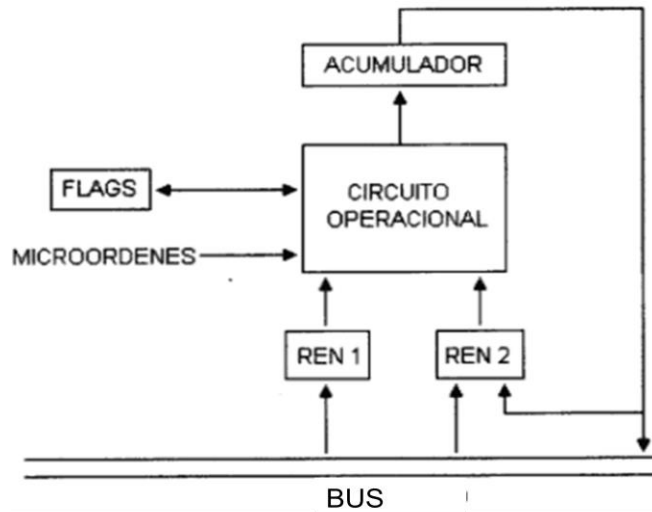
$AND = 2^q$

Entradas = $2^q \cdot 2$

Unidad Aritmética y Lógica: Se compone de una unidad capaz de ejecutar todo el surtido de instrucciones del calculador o de varias unidades funcionales u operadores, cada uno especializado en la ejecución de una o varias clases de operaciones.

La ALU está formada por los siguientes elementos:

1. Operadores
2. Registros de entrada (REN)
3. Registro acumulador
4. Registro de estado (flags, indicadores de estado)



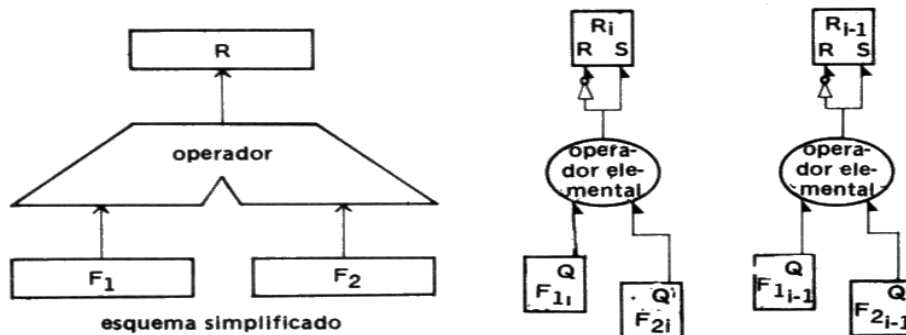
Los operadores pueden trabajar en serie o en paralelo. Y pueden ser de carácter Combinacional, ejecutan las operaciones en una sola fase, o de carácter Secuencial, ejecutan las operaciones en varias fases gobernadas por impulsos distribuidos por un dispositivo de control y que poseen dispositivo de memorización para los resultados parciales.

Los indicadores suministran información acerca de la última operación realizada. Hay indicadores de Error y del estado del acumulador.

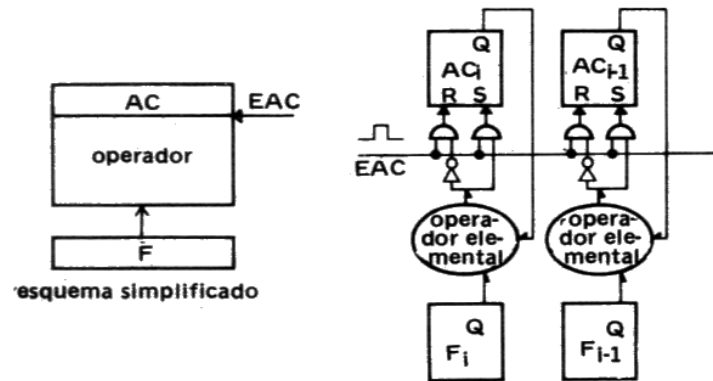
Los operandos son cargados en los REN (registros de entrada), y los resultados parciales en el acumulador.

ALU Elemental: Existen 2 montajes posibles de los operadores cuando hablamos de operaciones entre bits del mismo peso:

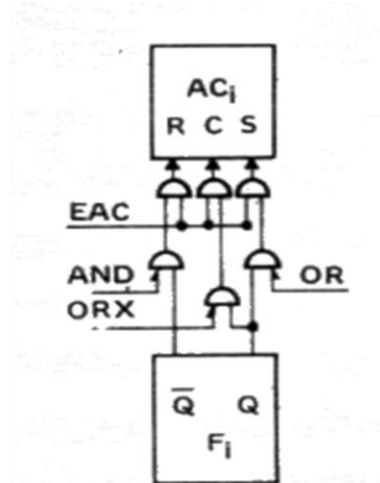
- Operadores Combinacionales: se montan entre dos registros fuentes para los operandos y un registro para el resultado



- Operadores con Acumulador: no existen más que un registro fuente para almacenar un operando, ya que el otro se carga al acumulador durante la ejecución de la operación. Cuando la salida de los operadores elementales queda establecida, un impulso EAC habilita el resultado en el acumulador.

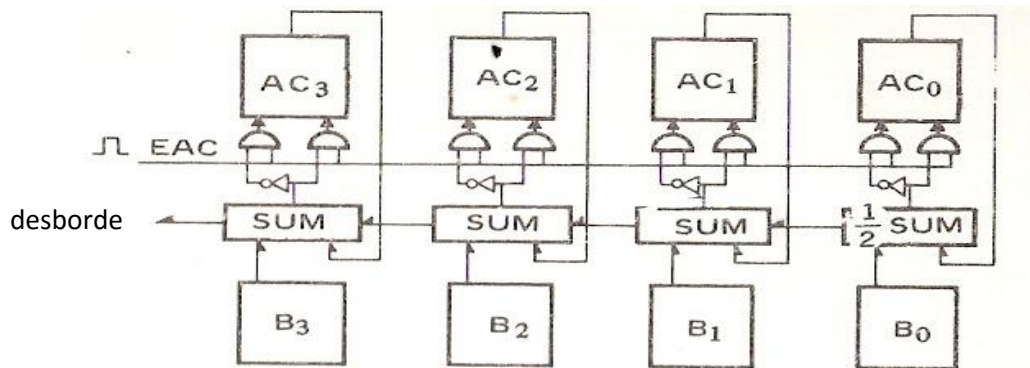


Operaciones Lógicas (con Acumulador):

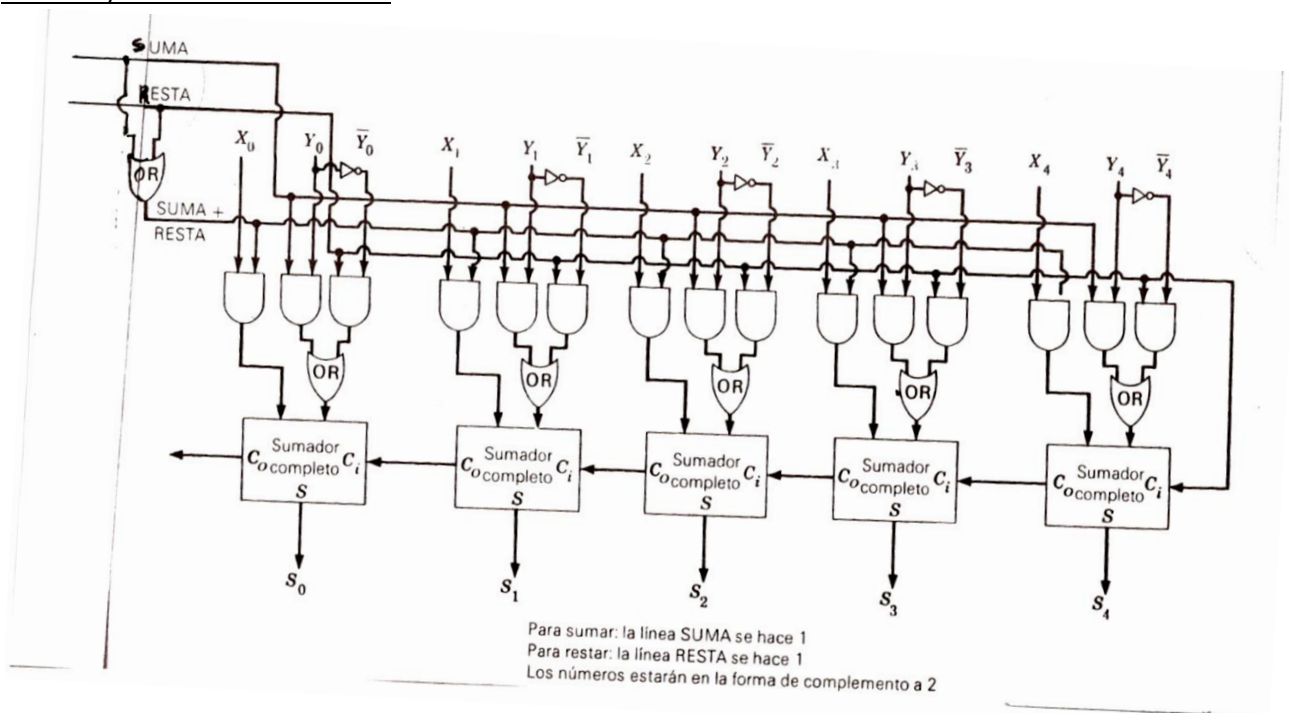


Elemento con operaciones Integradas

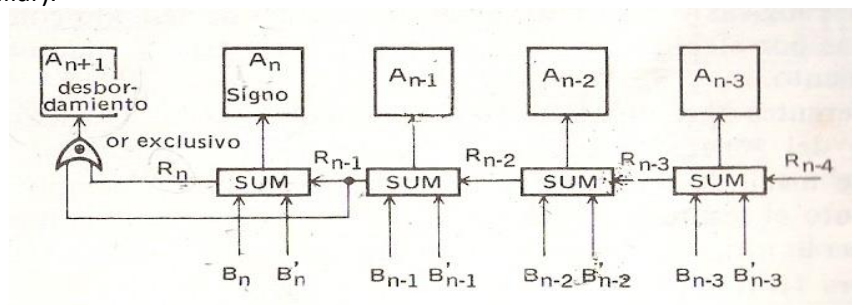
Sumador Binario en Paralelo para números sin Signo: Es básicamente una cadena de etapas de sumador (la primera puede ser un medio-sumador). Cada etapa transmite su desbordamiento a la siguiente.



Sumador/Sustractor en Paralelo:



Suma/Sustracción de números binarios con signo: Restar un número de otro equivale a sumarle su opuesto (el cual calculamos por complementación), entonces todo se reduce a la suma algebraica. En este caso solo trabajaremos con complemento a 2 (ya que el complemento restringido se opera de forma similar).



Suponemos una ALU que procesa número representados por n bits, con el bit de signo situado en la primera posición a la izquierda (n). Asociaremos al acumulador un bit que activara el indicador de desbordamiento. El indicador solo se activara en los siguientes casos:

- Cuando no hay arrastre de orden $n-1$ pero hay arrastre de orden n .
- Cuando hay arrastre de orden $n-1$ pero no hay arrastre de orden n .

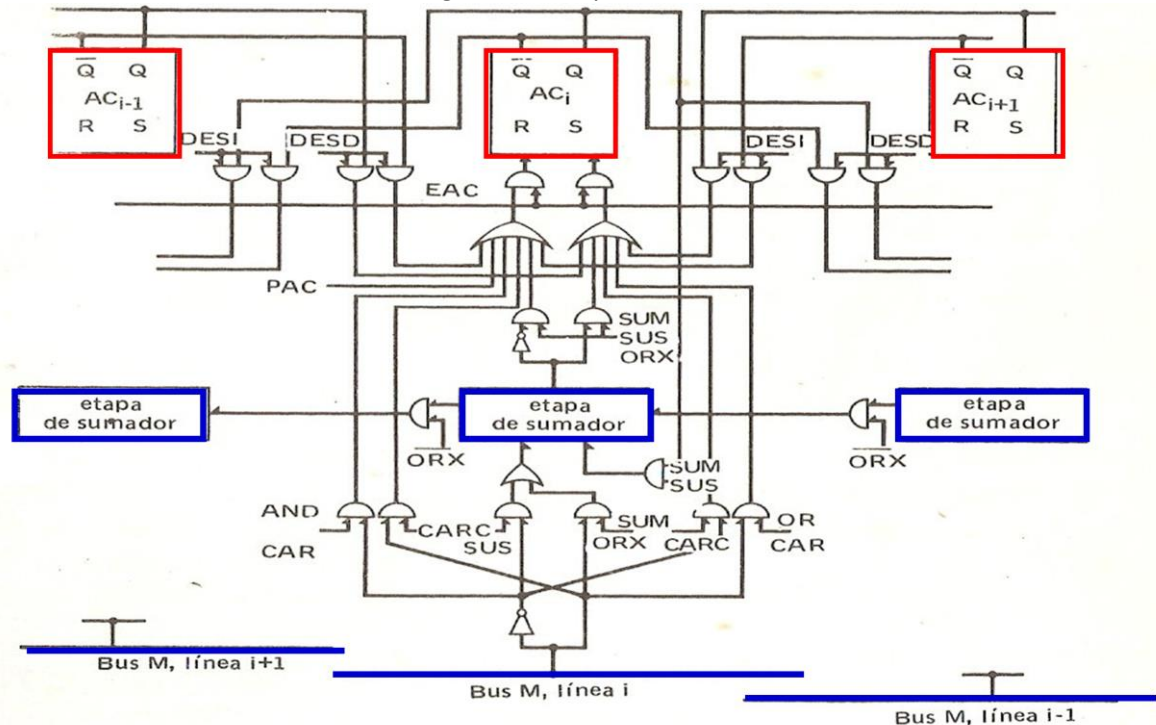
CONDICIÓN DE SIGNO	Los 2 Op. Son positivos. El resultado es positivo.	1 Op. Es positivo y 1 Op. Es negativo. No puede haber desbordamiento.	Los 2 Op. Son negativos. El resultado es negativo.
Repres. inicial	0 _ _ _ _ 0 _ _ _ _	1 _ _ _ _ 0 _ _ _ _	1 _ _ _ _ 1 _ _ _ _

Res. Si no hay arrastre de la etapa n-1	0 _ _ _ _ _ Válido res. positivo	1 _ _ _ _ _ Válido res. negativo	10 _ _ _ _ _ No válido por desbordamiento
Res. Si hay arrastre de la etapa n-1	1 _ _ _ _ _ No Válido res. negativo Desbordamiento	10 _ _ _ _ _ Válido res. positivo dejando de lado el desbordamiento	11 _ _ _ _ _ Válido res. negativo dejando de lado el desbordamiento

Elemento de ALU (Abacus): la ALU de Abacus está montada entre un Bus fuente M, donde se mantienen los niveles lógicos representativos del segundo operando, un acumulador AC que mantiene los niveles lógicos del primer operando hasta que la señal de muestreo de EAC introduce el resultado en el acumulador.

Consta de un sumador en paralelo y un conjunto de puertas para ejecutar las operaciones lógicas y distribuir las informaciones de acuerdo con las operaciones aritméticas por efectuar. Las diferentes operaciones están gobernadas por señales lógicas procedentes del secuenciador.

El acumulador está montado como registro de desplazamiento.



Operaciones que permite		Indicadores
CAR- Carga al acumulador	AND – Intersección lógica	DESB – Desbordamiento
CARC – Carga con complementación	OR – Reunión lógica	S – Signo
SUM – Suma (AC+Bus M)	ORX – OR exclusivo	CE – Indicador de cero
SUS – Resta (AC-Bus M)	DESI – Desplaza a la izquierda	
PAC – Puesta a Cero	DESD – Desplaza a la derecha	

Multiplicador por Suma-Desplazamiento:

Consideremos el siguiente ejemplo numérico:

11001	~	multiplicando
10101		multiplicador
11001		productos parciales
00000		
11001		
00000		
11001		producto
1000001101		

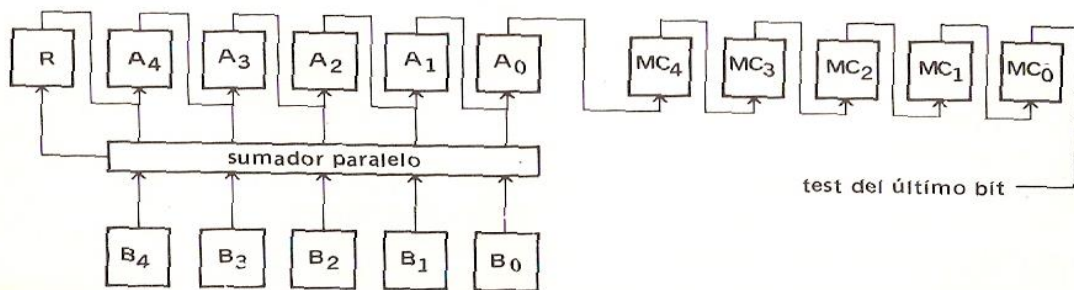
Los productos parciales son iguales al multiplicando si el bit correspondiente del multiplicador es igual a 1, son nulos en caso contrario. Por lo tanto la primera operación consiste en verificar sucesivamente cada bit del multiplicador.

La ALU capaz de realizar una multiplicación debe tener un anexo al acumulador llamado "Multiplicador-Cociente" (MC) que contiene el multiplicador en el caso de la multiplicación y el cociente en el caso de la división.

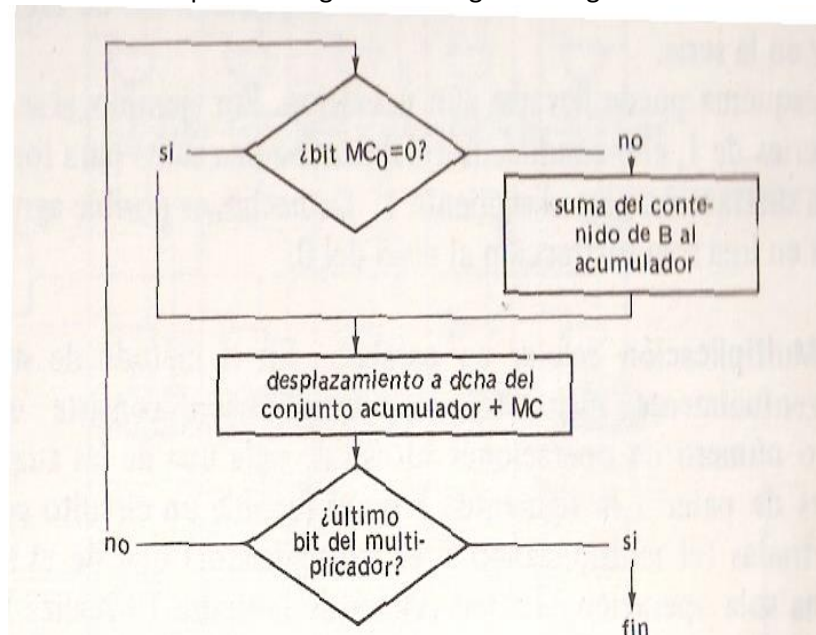
El resultado de la multiplicación ocupa el doble de la longitud (AC + MC).

La operación se realiza:

1. Carga del multiplicador en el acumulador
2. Desplazamiento a derecha del contenido del conjunto AC-MC (Hasta que AC quede en 0)
3. Carga del multiplicando en B.



4. Luego se continua la operación siguiendo el siguiente algoritmo:

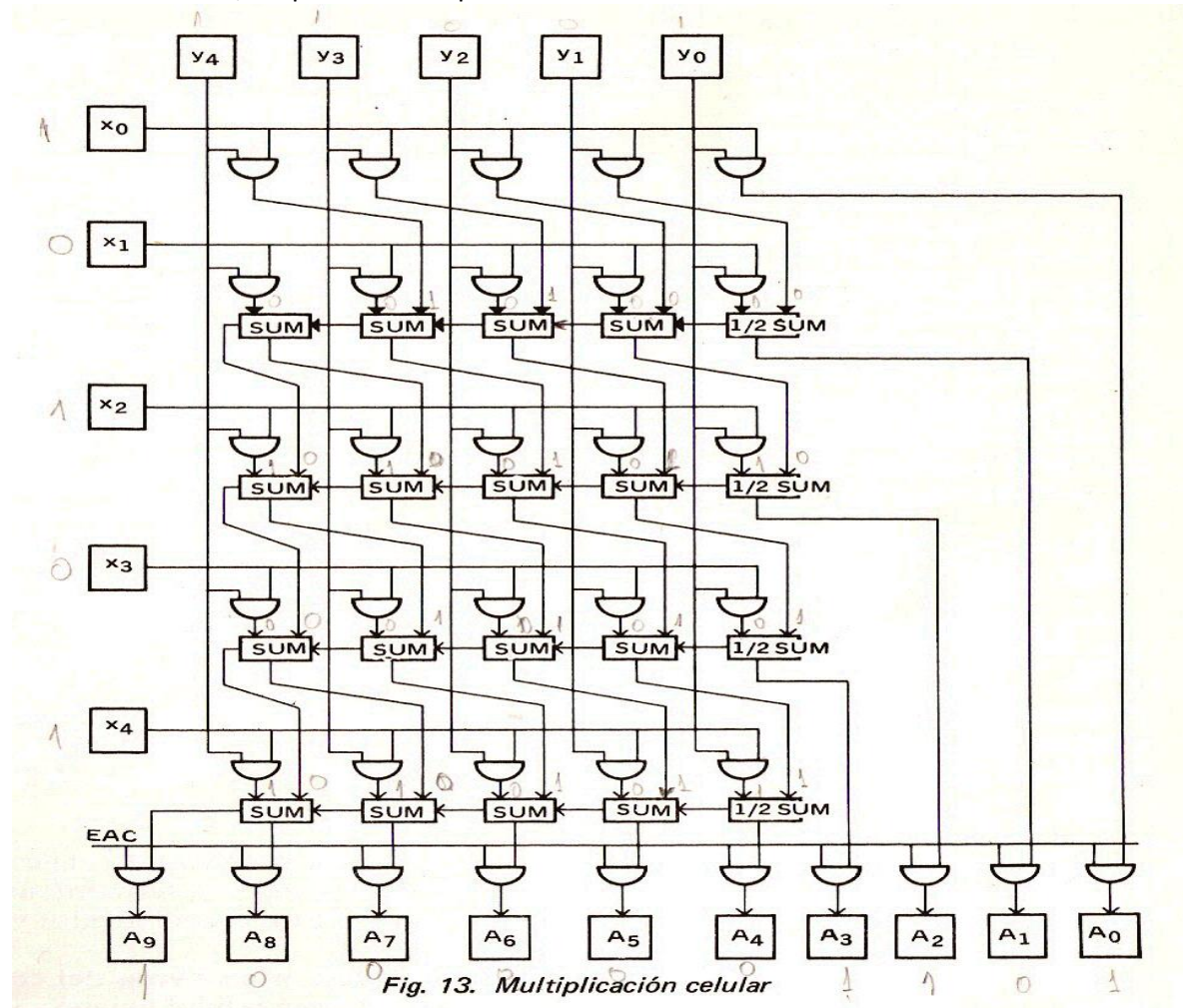


Multiplicación Celular en Paralelo: Consiste en un circuito Combinacional con dos entradas (multiplicando Y y multiplicador X) que da como resultado la salida A en una sola operación.

Exceptuando a la fila 0 y columna 0, cada célula está compuesta por una compuerta AND y un Sumador Completo.

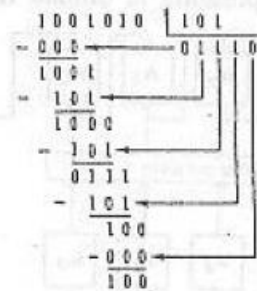
Fijando nuestra atención en la Célula:

- La compuerta AND está cerrada si $x_i=0$, en ese caso el sumador se limita a transmitir a la fila inferior el resultado parcial obtenido desplazándolo una posición.
- Si la compuerta AND se abre es sumador adiciona el resultado del producto parcial $x_i y_j$ ($x_i=1 \rightarrow y_j$) al resultado parcial obtenido en la fila superior, transmitiendo el resultado a la fila inferior, desplazando una posición.



División por Sustracción-Desplazamiento:

Partamos de un ejemplo numérico:

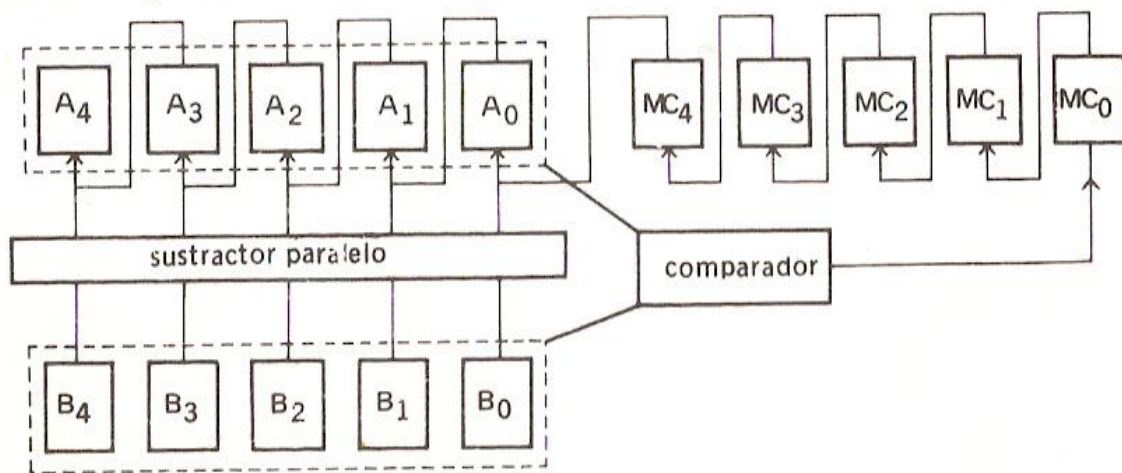


En la división se resta del dividendo el divisor o cero, dependiendo de si el correspondiente bit del cociente vale 1 o 0.

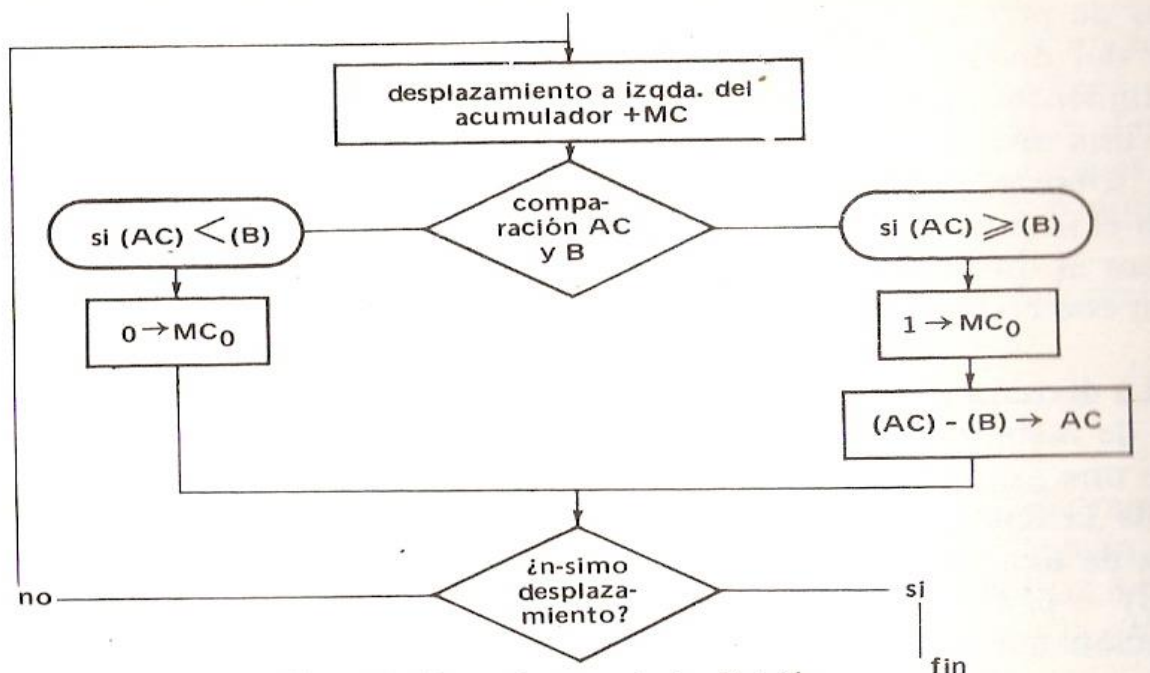
Es necesario incorporar la operación de comparación entre los bits de mayor peso del dividendo y del divisor a fin de determinar el bit correspondiente del cociente.

La operación:

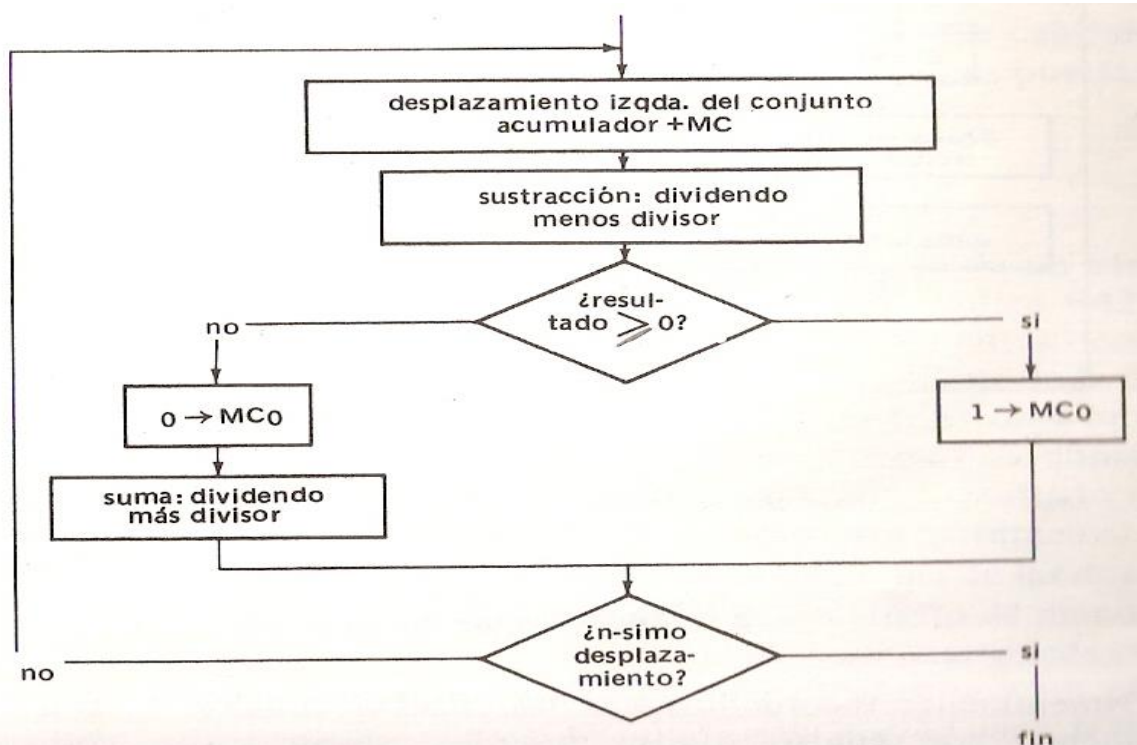
1. Carga del dividendo en el acumulador.
2. Desplazamiento a derecha del AC + MC (Puesta a cero del AC).
3. Carga del divisor en B.



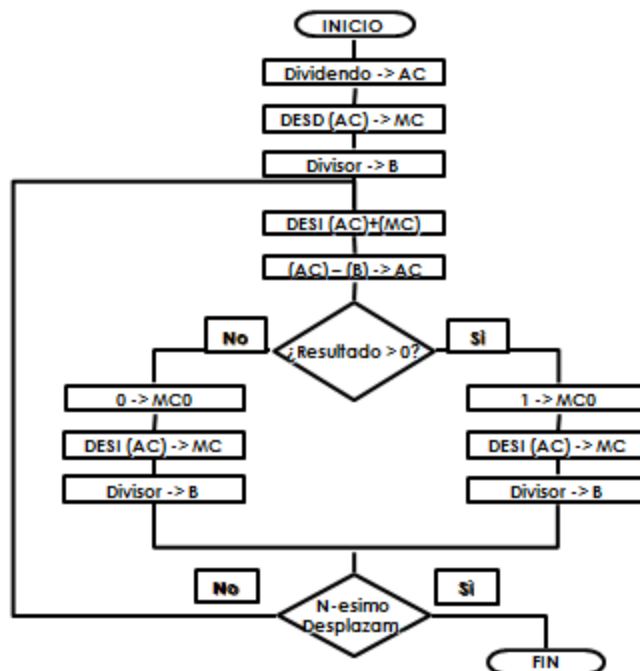
4. Luego se sigue el siguiente Algoritmo(con comparador):



Una alternativa, en caso de no tener dispositivo comparador (debido a su alto precio), es realizar la división con Restauración: la comparación se realiza intentando restar el divisor del dividendo y comparando el signo del resultado. Como la ALU pierde el valor antiguo del dividendo, es necesario restaurarlo, añadiendo el divisor al dividendo:



División sin restauración: corresponde a un algoritmo que permite ahorrar la fase de restauración después de una sustracción con resultado negativo, lo que supone una ganancia en tiempo.



Suma y Sustracción con coma flotante: Las operaciones se llevan a cabo sobre números normalizados, esto es, sobre números cuyo valor de exponente está ajustado para que la mantisa tenga el mayor número posible de dígitos significativos. Por consiguiente, los operadores en

aritmética flotante deben, no solamente ejecutar la operación correspondiente, sino también normalizar el resultado obtenido.

El proceso se divide en 3 fases:

1. *Alinear las mantisas* con objeto de enfrentar los bits de igual peso. Supongamos a sumar los números $A = 12000 \times 10^{-1}$ y $B = 80.000 \times 10^{-2}$, que se escriben, respectivamente:

A

+	0	0	1	2	0	0
---	---	---	---	---	---	---

B

+	2	8	0	0	0	0
---	---	---	---	---	---	---

No puede operarse directamente la suma de las mantisas; se comparan los exponentes y se detecta que el primero de los números es más pequeño; sobre este número se ejecuta una doble operación: *incremento del exponente en una unidad y desplazamiento de la mantisa una posición a la derecha*. De nuevo se comparan y se recomienza hasta obtener identidad de exponentes. En ese momento puede efectuarse la suma de las mantisas:

A

+	2	0	0	0	1	2
---	---	---	---	---	---	---

B

+	2	8	0	0	0	0
---	---	---	---	---	---	---

2. *Ajustar los signos*. Consideremos la sustracción $A - B$. En la representación en valor absoluto y signo, se deberán primeramente comparar las mantisas de A y B, comprobar que la de B es superior a la de A y deducir de ello que la operación pertinente es sustraer la mantisa de A de la mantisa de B y al resultado darle el signo menos:

A

+	2	0	0	0	1	2
---	---	---	---	---	---	---

 MB

8	0	0	0	0
---	---	---	---	---

- B

+	2	8	0	0	0	0
---	---	---	---	---	---	---

 - MA

0	0	0	1	2
---	---	---	---	---

-	2	7	9	9	8	8
---	---	---	---	---	---	---

Debe tomarse así en cuenta en esta fase la clase de operación (suma o sustracción), el signo de los operandos y el resultado de la comparación de mantisas. Nótese que el problema se simplifica si el conjunto signo-mantisa fuera representado en complementación cuando el número sea negativo.

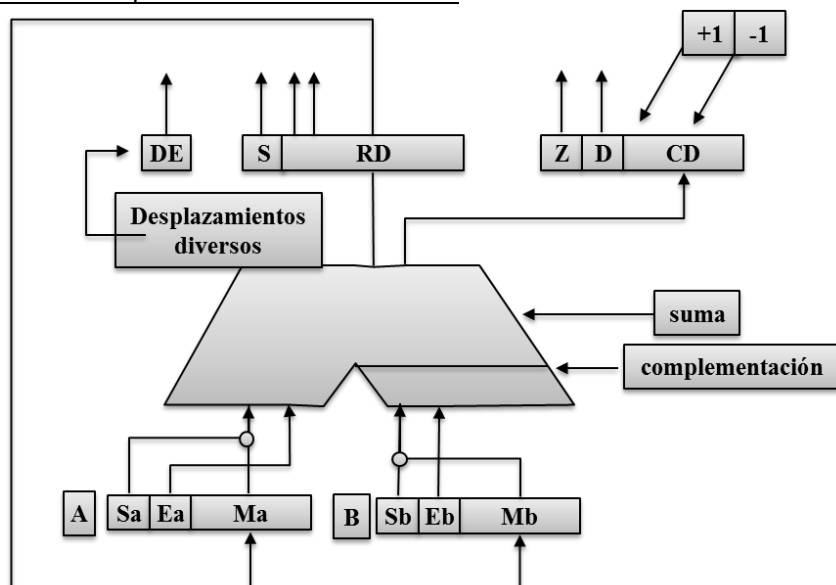
3. *Normalización del resultado*. Puede que el resultado bruto de la operación no esté normalizado, por dos razones:
 - hay desbordamiento de capacidad en la suma, en cuyo caso es necesario desplazar a mantisa una posición a la derecha, aumentando al tiempo en una unidad el exponente.
 - la diferencia de las mantisas implica la formación de un número n de ceros a la izquierda del resultado, en cuyo caso es necesario desplazar la mantisa n posiciones a la izquierda del resultado simultáneamente.

Veamos un ejemplo de suma en ambos casos:

1º Operando	+	2	8	0	0	0	0
2º Operando	+	2	8	0	0	0	0
Operación		1	6	0	0	0	0
Normalización	+	3	1	6	0	0	0

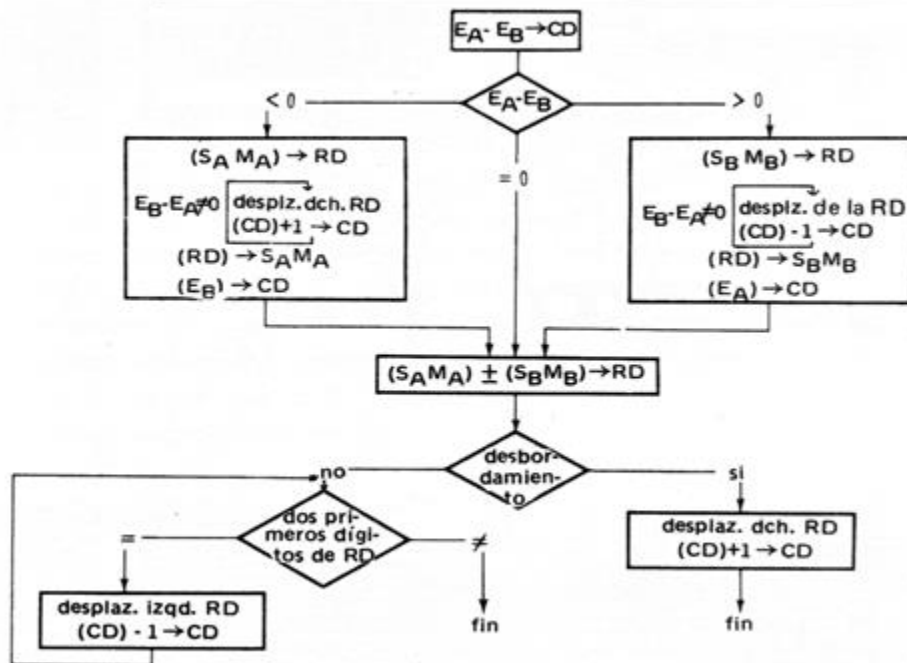
1º Operando	+	2	8	0	0	0	0
2º Operando	+	2	7	9	9	9	9
Operación			0	0	0	0	1
Normalización	+	-4	1	0	0	0	0

Funcionamiento de un Operador con coma flotante:



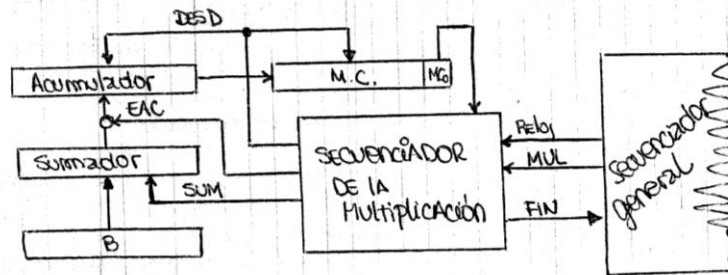
FASES:

1. Comparación de los exponentes
2. Alineamiento de las mantisas
3. Suma-sustracción de las mantisas
4. Normalización



Secuenciador Aritmético: En las maquinas complejas, el control de los operadores aritméticos de funcionamiento secuencial esta frecuentemente descentralizado y cada operador posee su órgano de Control. En estos casos el secuenciador general (explicado mejor más adelante) envía una orden especificando el tipo de operación a ejecutar y los batidos de reloj central, si el operador no tuviese uno propio. El operador se encarga entonces del secuenciamiento de la operación aritmética. Una vez terminada la operación, el operador envía al secuenciador central una señal de fin de operación. El secuenciador de un operador secuencial puede poseer su propio decodificador de órdenes, su contador de fases, sus Biestables de estado e incluso su propio reloj.

Un ejemplo es el operador de multiplicación por suma-desplazamiento:



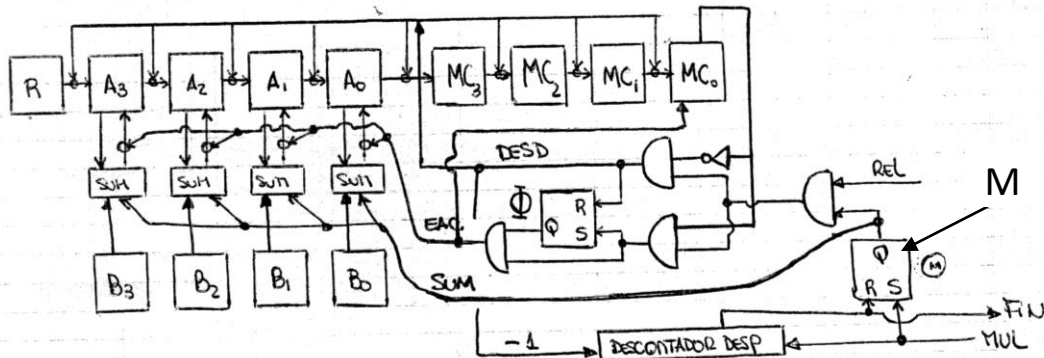
Recibe del secuenciador general la orden de multiplicación y los batidos de reloj. Envía al sumador la orden de SUM, la validación de la suma EAC y la orden de desplazamiento a derecha DESD.

Recibe del último Biestable del MC una señal indicativa de estado. Al finalizar la multiplicación ese secuenciador enviara al secuenciador general una señal de fin de operación.

Por dentro:

- El sumador permanece activado mientras dure la multiplicación.
- La duración máxima de la suma es 2 batidos reloj (en ausencia de sum, MC=0, se produce un desplazamiento a cada batido de reloj).
- MUL carga el descontador de desp. Con una cantidad igual al número de dígitos de los operandos. Posiciona el Biestable M que estará en 1 durante toda la multiplicación.

- Al primer batido de reloj siguiente al posicionamiento de M comienza la operación:
Si $MC_0=0$ entonces: DESD y decremento del descontador
Si $MC_0=1$ entonces: SUM (dividida en 2 batidos) y DESD
Este paso se repite hasta que el descontador valga 0.
- Cuando el descontador vale 0, se genera un impulso que pone a cero M y una señal al secuenciador central de fin de operación.



Bibliografía:

Filminas del Ing. Vicente Martin

Filminas del Ing. Gustavo Maurokefalidis

Filminas de la Ing. Liliana Cuenca Pletsch

Apuntes realizados por los previamente mencionados y el Ing. Marcelo Karanik

Carpeta/Resumen de Juan Carcaño (esto me facilito muchísimo la vida, gracias)

Stallings, W, Organización y Arquitectura de Computadores. Ed. Prentice Hall. 4ª

Edición. 1.997

Meinadier, J. P., Estructura y Funcionamiento de los Computadores Digitales. Ed. AC.

1.975

Quiroga, Arquitectura de Computadoras, 1ra Edición

Martinez Dura, Boluda Grau, Perez Solano, Estructura de Computadores y Periféricos

Ginzburg, M.C. La PC por dentro, modelo de funcionamiento. Ed. Biblioteca Técnica

Superior - Bs. As. 1.996

Un libro muy bueno que no puedo identificar, que esta escaneado sin las primeras páginas y cuyo nombre de archivo es "capitulo1-12.pdf"

Apunte de Arquitectura de Computadoras de la UNCa